School of Computer Science Engineering and Information Systems
BCSE302L – Database Systems | Course Project

EMPLOYEE PAYROLL MANAGEMENT SYSTEM

Review II – Database Implementation

Submitted by

Krishiv Rajkumar – 24BDE0132
Asher Mathew – 24BDE0128
Sai Praneeth – 24BYB0143

Under the guidance of
Dr Rubimalaya R

October 2026

Contents

Contents 2
1. Project Title and Team Details 4
2. Introduction and Problem Statement 5
2.1 Introduction 5
2.2 Problem Statement 5
2.3 Objectives 5
2.4 Users and Roles 5
3. Functional Requirements 7
4. ER Diagram 8
5. Relational Schema 9
5.1 Relational Schema (text notation) 10
6. Table Creation and Constraints 11
6.1 Constraint Summary 11
6.2 DDL – sql/01_schema.sql 11
6.3 Data Population 13
6.4 DML Demonstration – INSERT, UPDATE and DELETE 17
7. SQL Queries and Outputs 19
Q1. Employee directory with department, designation and manager 19
Q2. Head-count and average basic salary of every department (including empty ones) 20
Q3. Employees whose basic salary is above the company average 20
Q4. Department-wise salary cost for the latest payroll month 21
Q5. Top 5 earners by net salary in the latest month (with rank) 21
Q6. Attendance summary of every employee for September 2026 22
Q7. Employees who lost pay (LOP) in any month, with the amount lost 22
Q8. Remaining 2026 leave balance of every active employee 23
Q9. Pending leave requests and the manager who must approve them 23
Q10. Managers with at least 3 direct reports 24
Q11. Itemised payslip lines of every employee for the latest month 24
Q12. Active employees who have not taken any approved leave in 2026 25
Q13. Month-by-month payroll totals and payment status 25
Q14. Departments whose average net salary is above the company average (latest month) 26
Q15. Age and tenure of every active employee (derived attributes) 26
Q16. Total paid out per salary component in the latest month 27
Q17. Employees who were both absent and on approved leave in September 27
Q18. Everyone with leadership responsibility: department heads or people managers 28
Q19. Employees who checked in after 09:35 more than 10 times since August 28
8. Procedures, Functions, Cursor and Triggers 30

8.1 Overview 30
8.2 Functions 30
8.3 Procedures and Cursors 33
8.4 Triggers 36
8.5 Calling Procedures from the Application 39
9. Real-World Transaction Demonstrations 41
9.1 Application and Database Connectivity 41
T1. Employee checks in 42
T2. Employee applies for leave 42
T3. Manager approves leave 43
T4. Payroll officer generates the monthly payroll 44
T5. Viewing an itemised payslip 45
T6. Paying the month's salaries 46
T7. Attempt to delete a paid payslip 47
T8. Employee resignation 48
T9. Running the report queries from the application 48
10. Conclusion 50

(If page numbers are missing, right-click the table and choose "Update Field".)

1. Project Title and Team Details

Item	Details
Project title	Employee Payroll Management System
Course	BCSE302L – Database Systems (Lab course project)
Team	Krishiv Rajkumar (24BDE0132), Asher Mathew (24BDE0128), Sai Praneeth (24BYB0143)
Guide	Dr Rubimalaya R
Database	PostgreSQL 17 hosted on Supabase (cloud); PL/pgSQL for procedures, functions, cursors and triggers
Application	React + Vite web app (deployed on Vercel) connected through the supabase-js client
Source code	github.com/LordFlyaway/DBS_proj
Sample company	All Caps Pvt. Ltd. – a fictional company whose 20 employees are named after musicians and athletes

2. Introduction and Problem Statement

2.1 Introduction

Every organisation has to pay its employees correctly and on time. A salary is not one fixed number: it depends on the designation and basic pay, allowances such as HRA and DA, deductions such as Provident Fund, Professional Tax and TDS, the days actually worked, and approved or unapproved leave. When this is managed in spreadsheets, calculations go wrong, records are duplicated, nothing stops invalid entries and there is no audit trail.

The Employee Payroll Management System stores organisation, employee, attendance, leave and salary-structure data in a relational database and generates monthly payroll and payslips from that data with stored procedures. Business rules are enforced inside the database through constraints, triggers and Row Level Security, and a role-based web application lets HR, payroll staff, managers and employees work with the data.

2.2 Problem Statement

Design and implement a centralised, consistent and secure database that (a) maintains employee and organisational data, (b) records daily attendance and leave, (c) defines salary components (allowances and deductions), and (d) computes each employee's monthly gross and net salary, including loss-of-pay for unpaid absence, while enforcing business rules with database constraints and triggers instead of manual checks.

2.3 Objectives

- A single source of truth for employees, departments and designations.
- Attendance and leave recording, with loss-of-pay calculated automatically.
- Configurable salary components – fixed amounts or a percentage of basic pay.
- One-step monthly payroll generation with itemised payslips, followed by salary payment.
- Integrity through PK/FK, NOT NULL, UNIQUE, CHECK and DEFAULT constraints, triggers and Row Level Security.
- Interactive access through a web application connected to the database.

2.4 Users and Roles

Role (demo login)	**What the role can do**
Admin – Daniel Dumile (HR Manager)	Manage employees, departments, designations and attendance; approve any leave; run payroll; run the SQL query catalogue.
Payroll – Jeffery Williams (Payroll Officer)	Maintain salary components; generate, pay and (if unpaid) delete payroll; run the SQL query catalogue.
Manager – Aubrey Graham (Engineering Manager)	See their team; approve or reject the team's leave; follow team attendance.
Employee – LeBron James (Sr. Software Engineer)	Check in and out; apply for and cancel leave; view own attendance and payslips.

Every role also sees its own dashboard, leave balance and payslips. Logins are e-mail/password accounts in Supabase Auth, linked to EMPLOYEE rows through `employee.auth_user_id` and authorised through `employee.app_role`.

3. Functional Requirements

ID	Requirement	Implemented by
FR1	Add, update and delete departments, including the department head.	DEPARTMENT; Departments page
FR2	Maintain designations with pay grade, basic salary and annual leave quota.	DESIGNATION; Designations page
FR3	Register employees with personal, PAN and bank details, department, designation, manager and login role.	EMPLOYEE; CHECK/UNIQUE constraints; trg_validate_manager
FR4	Record attendance – one row per employee per day; employees check themselves in and out.	ATTENDANCE; UNIQUE (emp_id, att_date); RLS policy
FR5	Employees apply for leave; managers approve or reject; approved leave appears in attendance.	LEAVE_REQUEST; process_leave; trg_leave_approved
FR6	Define allowances and deductions as fixed amounts or a percentage of basic.	SALARY_COMPONENT; fn_component_amount
FR7	Generate the month's payroll for all active employees with LOP, gross and net salary.	generate_monthly_payroll (cursor); fn_working_days; fn_lop_days
FR8	Keep an itemised payslip (amount of every component).	PAYROLL_DETAIL (M:N bridge)
FR9	Pay salaries: mark payroll Paid with date, mode and transaction reference.	pay_monthly_payroll (cursor FOR UPDATE)
FR10	Prevent invalid operations – duplicate payroll, editing paid payroll, self-approval, exceeding leave balance.	Constraints, procedures and triggers
FR11	Handle employee exits – reassign their team, clear department headship, reject pending leave.	trg_employee_exit
FR12	Management reports: department cost, attendance summary, leave balance, top earners and others.	19 SQL queries saved as views
FR13	Role-based web application connected to the database.	React app + supabase-js + Row Level Security

4. ER Diagram

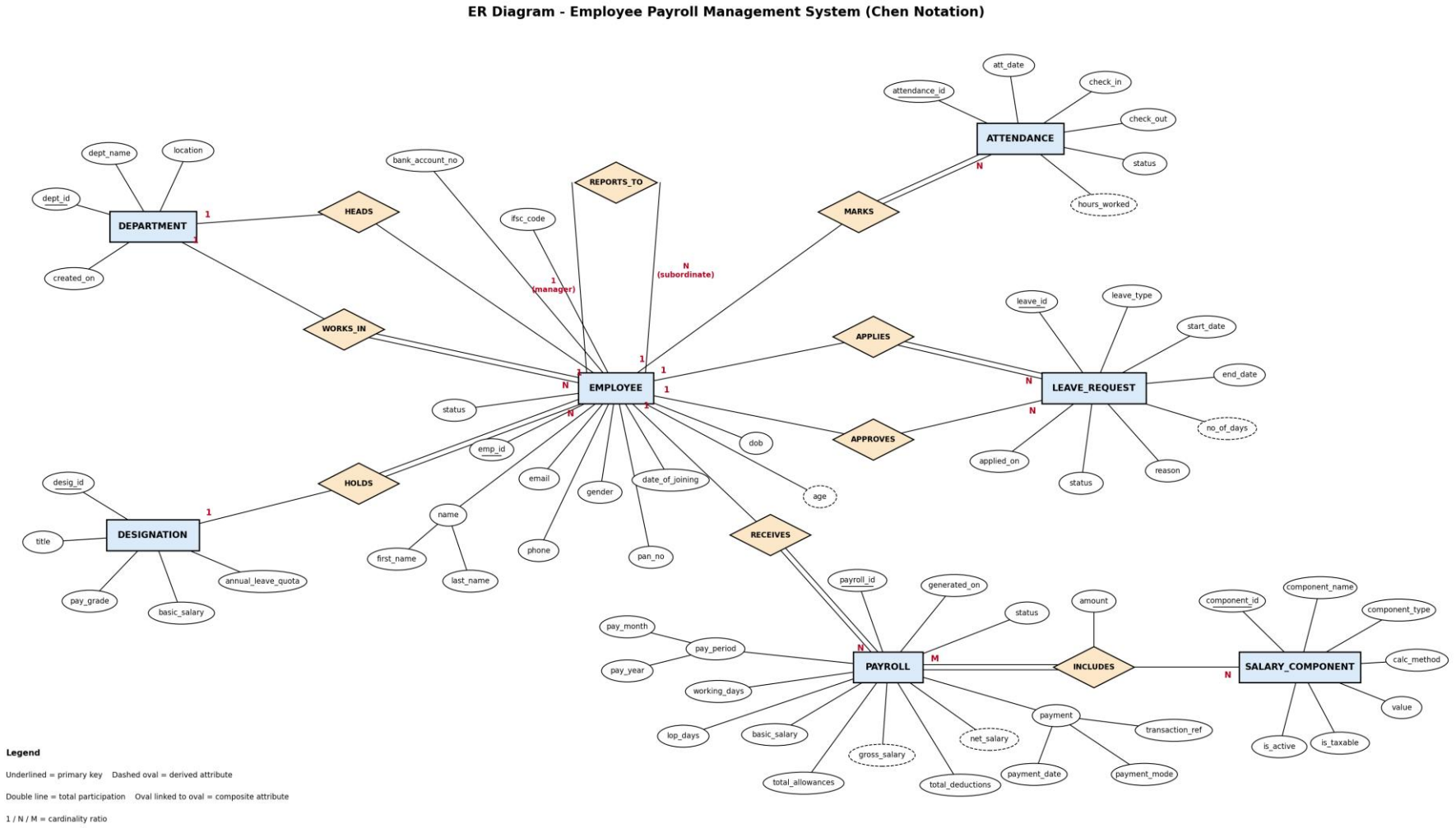


Figure 1 – ER diagram (Chen notation): 7 entities and 9 relationships, including the recursive REPORTS_TO, the 1:1 HEADS and the M:N INCLUDES relationship.

5. Relational Schema

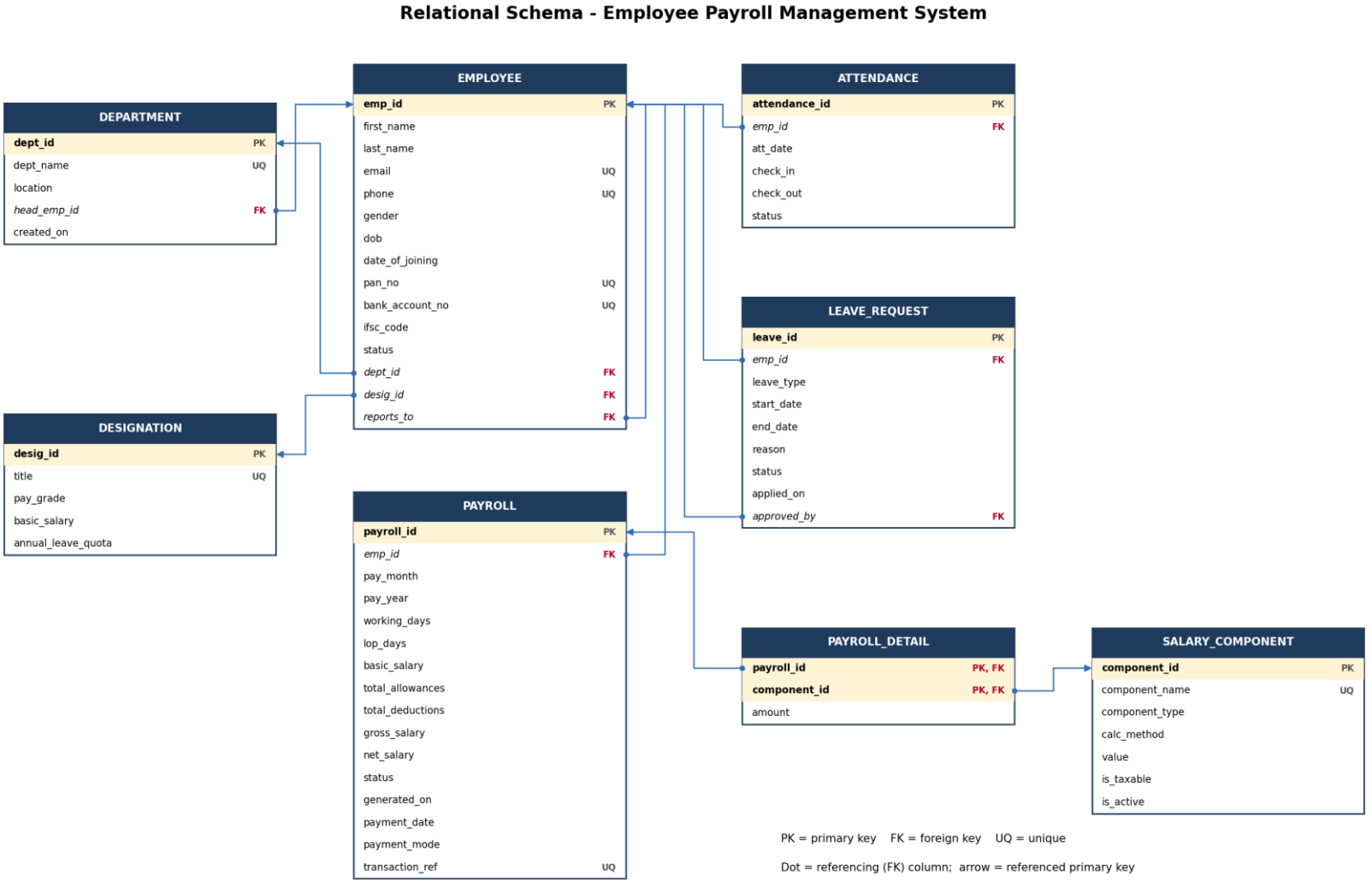


Figure 2 – Relational schema: 8 tables with primary and foreign keys.

5.1 Relational Schema (text notation)

Primary keys are **bold and underlined**; foreign keys are *italic*.

DEPARTMENT (**dept_id**, dept_name, location, *head_emp_id*, created_on)

DESIGNATION (**desig_id**, title, pay_grade, basic_salary, annual_leave_quota)

EMPLOYEE (**emp_id**, first_name, last_name, email, phone, gender, dob, date_of_joining, pan_no, bank_account_no, ifsc_code, status, *dept_id*, *desig_id*, *reports_to*, app_role, auth_user_id)

ATTENDANCE (**attendance_id**, *emp_id*, att_date, check_in, check_out, status)

LEAVE_REQUEST (**leave_id**, *emp_id*, leave_type, start_date, end_date, reason, status, applied_on, *approved_by*)

SALARY_COMPONENT (**component_id**, component_name, component_type, calc_method, value, is_taxable, is_active)

PAYROLL (**payroll_id**, *emp_id*, pay_month, pay_year, working_days, lop_days, basic_salary, total_allowances, total_deductions, gross_salary, net_salary, status, generated_on, payment_date, payment_mode, transaction_ref)

PAYROLL_DETAIL (***payroll_id***, ***component_id***, amount)

Change since Review I: two columns were added to EMPLOYEE for the web application – `app_role` (Admin, Payroll, Manager or Employee) and `auth_user_id` (the employee's Supabase login, UNIQUE). Keeping them in EMPLOYEE keeps the design at 8 tables.

All tables are in Third Normal Form: department and designation details are stored once and referenced by key, salary rules are separate from payslip line items, and the only non-key attribute of the composite-key table PAYROLL_DETAIL (amount) depends on the whole key. Age, hours worked and leave days are derived in queries and are not stored; gross and net salary are stored on purpose as a monthly snapshot so that old payslips do not change when salary rules change.

6. Table Creation and Constraints

The database is created by `sql/01_schema.sql`. Each table and the business rule behind each constraint is summarised below, followed by the complete DDL.

6.1 Constraint Summary

Table	Keys	NOT NULL / UNIQUE / CHECK / DEFAULT
DEPARTMENT	PK dept_id; FK head_emp_id (UNIQUE) → employee	dept_name UNIQUE NOT NULL; created_on DEFAULT CURRENT_DATE
DESIGNATION	PK desig_id	title UNIQUE; pay_grade IN (G1–G5); basic_salary > 0; annual_leave_quota DEFAULT 24, 0–60
EMPLOYEE	PK emp_id; FK dept_id, desig_id, reports_to (self)	email, phone, pan_no, bank_account_no, auth_user_id UNIQUE; phone ^[6-9][0-9]{9}$; PAN ^[A-Z]{5}[0-9]{4}[A-Z]$; IFSC format; gender IN (M,F,O); status DEFAULT 'Active'; app_role DEFAULT 'Employee'; at least 18 at joining; cannot report to self
ATTENDANCE	PK attendance_id; FK emp_id (CASCADE)	UNIQUE (emp_id, att_date); status IN (Present, Absent, Half-Day, Leave) DEFAULT 'Present'; check_out > check_in
LEAVE_REQUEST	PK leave_id; FK emp_id (CASCADE), approved_by (SET NULL)	leave_type IN (Casual, Sick, Earned, Unpaid); end_date ≥ start_date; status DEFAULT 'Pending'; approver ≠ applicant
SALARY_COMPONENT	PK component_id	component_name UNIQUE; type IN (Allowance, Deduction); calc_method IN (Fixed, Percent); value ≥ 0 and ≤ 100 for Percent; is_taxable, is_active DEFAULT TRUE
PAYROLL	PK payroll_id; FK emp_id (RESTRICT)	UNIQUE (emp_id, pay_month, pay_year); month 1–12; lop_days ≤ working_days; net_salary ≥ 0; status DEFAULT 'Generated'; Paid ⇒ payment_date and mode present; transaction_ref UNIQUE
PAYROLL_DETAIL	PK (payroll_id, component_id); FK both (CASCADE / RESTRICT)	amount ≥ 0

6.2 DDL – sql/01_schema.sql

```
-- ========================================================================
--  Employee Payroll Management System
--  BCSE302L Database Systems - Course Project
--  Target DBMS : PostgreSQL 15+ (hosted on Supabase)
--  File        : 01_schema.sql  (table creation + constraints)
-- ========================================================================

-- Drop in reverse dependency order so the script can be re-run
DROP TABLE IF EXISTS payroll_detail   CASCADE;
DROP TABLE IF EXISTS payroll          CASCADE;
DROP TABLE IF EXISTS salary_component CASCADE;
DROP TABLE IF EXISTS leave_request    CASCADE;
DROP TABLE IF EXISTS attendance       CASCADE;
DROP TABLE IF EXISTS employee         CASCADE;
DROP TABLE IF EXISTS designation      CASCADE;
DROP TABLE IF EXISTS department       CASCADE;

-- ------------------------------------------------------------------------
-- 1. DEPARTMENT : organisational units of the company
-- ------------------------------------------------------------------------
CREATE TABLE department (
    dept_id      SERIAL        PRIMARY KEY,
    dept_name    VARCHAR(50)   NOT NULL UNIQUE,
    location     VARCHAR(50)   NOT NULL,
    head_emp_id  INT,                              -- FK added after EMPLOYEE exists
    created_on   DATE          NOT NULL DEFAULT CURRENT_DATE
```

```
);

-- ---------------------------------------------------------------------
-- 2. DESIGNATION : job titles and their basic pay
-- ---------------------------------------------------------------------
CREATE TABLE designation (
    desig_id     SERIAL        PRIMARY KEY,
    title        VARCHAR(50)   NOT NULL UNIQUE,
    pay_grade    CHAR(2)       NOT NULL CHECK (pay_grade IN ('G1','G2','G3','G4','G5')),
    basic_salary NUMERIC(10,2) NOT NULL CHECK (basic_salary > 0),
    annual_leave_quota INT     NOT NULL DEFAULT 24 CHECK (annual_leave_quota BETWEEN 0 AND 60)
);

-- ---------------------------------------------------------------------
-- 3. EMPLOYEE : core entity; reports_to is a recursive (self) FK
-- ---------------------------------------------------------------------
CREATE TABLE employee (
    emp_id          SERIAL        PRIMARY KEY,
    first_name      VARCHAR(30)   NOT NULL,
    last_name       VARCHAR(30)   NOT NULL,
    email           VARCHAR(100)  NOT NULL UNIQUE CHECK (email LIKE '%_@_%._%'),
    phone           CHAR(10)      NOT NULL UNIQUE CHECK (phone ~ '^[6-9][0-9]{9}$'),
    gender          CHAR(1)       NOT NULL CHECK (gender IN ('M','F','O')),
    dob             DATE          NOT NULL,
    date_of_joining DATE          NOT NULL DEFAULT CURRENT_DATE,
    pan_no          CHAR(10)      NOT NULL UNIQUE CHECK (pan_no ~ '^[A-Z]{5}[0-9]{4}[A-Z]$'),
    bank_account_no VARCHAR(18)   NOT NULL UNIQUE,
    ifsc_code       CHAR(11)      NOT NULL CHECK (ifsc_code ~ '^[A-Z]{4}0[A-Z0-9]{6}$'),
    status          VARCHAR(10)   NOT NULL DEFAULT 'Active'
                                  CHECK (status IN ('Active','On Leave','Resigned','Terminated')),
    dept_id         INT           NOT NULL REFERENCES department(dept_id),
    desig_id        INT           NOT NULL REFERENCES designation(desig_id),
    reports_to      INT           REFERENCES employee(emp_id) ON DELETE SET NULL,
    app_role        VARCHAR(10)   NOT NULL DEFAULT 'Employee'
                                  CHECK (app_role IN ('Admin','Payroll','Manager','Employee')),
    auth_user_id    UUID          UNIQUE,              -- login account (Supabase auth.users.id)
    CONSTRAINT chk_emp_age     CHECK (date_of_joining >= dob + INTERVAL '18 years'),
    CONSTRAINT chk_not_own_mgr  CHECK (reports_to IS NULL OR reports_to <> emp_id)
);

-- Department head is an employee (1:1 "heads" relationship)
ALTER TABLE department
    ADD CONSTRAINT fk_dept_head
    FOREIGN KEY (head_emp_id) REFERENCES employee(emp_id) ON DELETE SET NULL;
ALTER TABLE department
    ADD CONSTRAINT uq_dept_head UNIQUE (head_emp_id);

-- ---------------------------------------------------------------------
-- 4. ATTENDANCE : one row per employee per working day
-- ---------------------------------------------------------------------
CREATE TABLE attendance (
    attendance_id SERIAL       PRIMARY KEY,
    emp_id        INT          NOT NULL REFERENCES employee(emp_id) ON DELETE CASCADE,
    att_date      DATE         NOT NULL,
    check_in      TIME,
    check_out     TIME,
    status        VARCHAR(8)   NOT NULL DEFAULT 'Present'
                               CHECK (status IN ('Present','Absent','Half-Day','Leave')),
    CONSTRAINT uq_att_emp_day UNIQUE (emp_id, att_date),
    CONSTRAINT chk_att_times  CHECK (check_out IS NULL OR check_in IS NULL OR check_out > check_in)
);

-- ---------------------------------------------------------------------
-- 5. LEAVE_REQUEST : applied by an employee, approved by a manager
-- ---------------------------------------------------------------------
CREATE TABLE leave_request (
    leave_id     SERIAL       PRIMARY KEY,
    emp_id       INT          NOT NULL REFERENCES employee(emp_id) ON DELETE CASCADE,
    leave_type   VARCHAR(10)  NOT NULL CHECK (leave_type IN ('Casual','Sick','Earned','Unpaid')),
    start_date   DATE         NOT NULL,
    end_date     DATE         NOT NULL,
    reason       VARCHAR(200),
```

```
    status       VARCHAR(10)  NOT NULL DEFAULT 'Pending'
                              CHECK (status IN ('Pending','Approved','Rejected')),
    applied_on   DATE         NOT NULL DEFAULT CURRENT_DATE,
    approved_by  INT          REFERENCES employee(emp_id) ON DELETE SET NULL,
    CONSTRAINT chk_leave_dates   CHECK (end_date >= start_date),
    CONSTRAINT chk_not_self_appr CHECK (approved_by IS NULL OR approved_by <> emp_id)
);

-- ---------------------------------------------------------------------
-- 6. SALARY_COMPONENT : master list of allowances and deductions
-- ---------------------------------------------------------------------
CREATE TABLE salary_component (
    component_id   SERIAL        PRIMARY KEY,
    component_name VARCHAR(40)   NOT NULL UNIQUE,
    component_type VARCHAR(10)   NOT NULL CHECK (component_type IN ('Allowance','Deduction')),
    calc_method    VARCHAR(10)   NOT NULL CHECK (calc_method IN ('Fixed','Percent')),
    value          NUMERIC(10,2) NOT NULL CHECK (value >= 0),
    is_taxable     BOOLEAN       NOT NULL DEFAULT TRUE,
    is_active      BOOLEAN       NOT NULL DEFAULT TRUE,
    CONSTRAINT chk_percent_range CHECK (calc_method <> 'Percent' OR value <= 100)
);

-- ---------------------------------------------------------------------
-- 7. PAYROLL : one salary record per employee per month (+ payment info)
-- ---------------------------------------------------------------------
CREATE TABLE payroll (
    payroll_id        SERIAL        PRIMARY KEY,
    emp_id            INT           NOT NULL REFERENCES employee(emp_id),
    pay_month         SMALLINT      NOT NULL CHECK (pay_month BETWEEN 1 AND 12),
    pay_year          SMALLINT      NOT NULL CHECK (pay_year BETWEEN 2000 AND 2100),
    working_days      SMALLINT      NOT NULL CHECK (working_days BETWEEN 0 AND 31),
    lop_days          NUMERIC(4,1)  NOT NULL DEFAULT 0 CHECK (lop_days >= 0),
    basic_salary      NUMERIC(10,2) NOT NULL CHECK (basic_salary >= 0),
    total_allowances  NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (total_allowances >= 0),
    total_deductions  NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (total_deductions >= 0),
    gross_salary      NUMERIC(10,2) NOT NULL DEFAULT 0,
    net_salary        NUMERIC(10,2) NOT NULL DEFAULT 0 CHECK (net_salary >= 0),
    status            VARCHAR(10)   NOT NULL DEFAULT 'Generated'
                                    CHECK (status IN ('Generated','Paid','On Hold')),
    generated_on      TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    payment_date      DATE,
    payment_mode      VARCHAR(10)   CHECK (payment_mode IN ('NEFT','IMPS','Cheque')),
    transaction_ref   VARCHAR(30)   UNIQUE,
    CONSTRAINT uq_payroll_emp_month UNIQUE (emp_id, pay_month, pay_year),
    CONSTRAINT chk_lop_le_days      CHECK (lop_days <= working_days),
    CONSTRAINT chk_paid_has_date    CHECK (status <> 'Paid' OR (payment_date IS NOT NULL AND
payment_mode IS NOT NULL))
);

-- ---------------------------------------------------------------------
-- 8. PAYROLL_DETAIL : M:N between PAYROLL and SALARY_COMPONENT
--    (line items of a payslip)
-- ---------------------------------------------------------------------
CREATE TABLE payroll_detail (
    payroll_id   INT           NOT NULL REFERENCES payroll(payroll_id) ON DELETE CASCADE,
    component_id INT           NOT NULL REFERENCES salary_component(component_id),
    amount       NUMERIC(10,2) NOT NULL CHECK (amount >= 0),
    PRIMARY KEY (payroll_id, component_id)
);

-- Helpful indexes on foreign keys used in joins
CREATE INDEX idx_emp_dept      ON employee(dept_id);
CREATE INDEX idx_emp_desig     ON employee(desig_id);
CREATE INDEX idx_att_emp_date  ON attendance(emp_id, att_date);
CREATE INDEX idx_leave_emp     ON leave_request(emp_id);
CREATE INDEX idx_payroll_month ON payroll(pay_year, pay_month);
```

6.3 Data Population

The sample data (`sql/04_seed_data.sql`) describes All Caps Pvt. Ltd. from August to October 2026. Every table has at least 10 meaningful rows. Attendance is generated for every working day; payroll

rows are not typed in by hand but produced by calling the payroll procedures, exactly as the application does.

Table	Rows	Contents
department	10	Departments in seven cities, six with a department head
designation	13	Job titles from Operations Associate (G1) to Director (G5)
employee	20	20 employees in a 3-level reporting hierarchy; one resigned in August
attendance	971	Every weekday from 1 Aug to 9 Oct 2026, with absences, half days and leave
leave_request	13	Approved, rejected and pending requests, including unpaid leave
salary_component	12	6 active allowances, 4 active deductions, 2 inactive components
payroll	39	August (20 employees, Paid) and September (19 employees, Generated)
payroll_detail	390	10 line items per payslip

DEPARTMENT

dept_id	dept_name	location	head_emp_id	created_on
1	Human Resources	Chennai	2	2012-06-01
2	Engineering	Bengaluru	4	2012-06-01
3	Finance	Chennai	3	2012-06-01
4	Sales	Mumbai	5	2013-04-15
5	Marketing	Mumbai	14	2014-02-10
6	Operations	Pune	NULL	2015-09-01
7	Customer Support	Hyderabad	NULL	2016-01-04
8	Quality Assurance	Bengaluru	NULL	2017-07-17
9	Legal	New Delhi	NULL	2018-03-12
10	Administration	Vellore	1	2012-06-01

DESIGNATION

desig_id	title	pay_grade	basic_salary	annual_leave_quota
1	HR Manager	G4	85000.00	24
2	Software Engineer	G2	45000.00	20
3	Senior Software Engineer	G3	65000.00	22
4	Engineering Manager	G4	95000.00	24
5	Accountant	G2	40000.00	20
6	Payroll Officer	G3	52000.00	22
7	Sales Executive	G1	30000.00	18
8	Sales Manager	G4	80000.00	24
9	Marketing Analyst	G2	42000.00	20
10	Operations Associate	G1	28000.00	18
11	Director	G5	150000.00	30
12	HR Executive	G2	38000.00	20
13	Legal Counsel	G3	70000.00	22

EMPLOYEE

emp_id	first_name	last_name	gender	date_of_joining	dept_id	desig_id	reports_to	app_role	status
1	Sachin	Tendulkar	M	2012-06-01	10	11	NULL	Admin	Active
2	Daniel	Dumile	M	2015-03-16	1	1	1	Admin	Active

emp_id	first_name	last_name	gender	date_of_joining	dept_id	desig_id	reports_to	app_role	status
3	Jeffery	Williams	M	2016-07-11	3	6	1	Payroll	Active
4	Aubrey	Graham	M	2014-01-20	2	4	1	Manager	Active
5	Michael	Jordan	M	2013-08-05	4	8	1	Manager	Active
6	LeBron	James	M	2019-07-01	2	3	4	Employee	Active
7	Cristiano	Ronaldo	M	2021-08-02	2	2	4	Employee	Active
8	Serena	Williams	F	2022-01-10	2	2	4	Employee	Active
9	Lionel	Messi	M	2020-11-16	8	2	4	Employee	Active
10	Simone	Biles	F	2019-04-08	3	5	3	Employee	Active
11	Rafael	Nadal	M	2018-09-03	3	5	3	Employee	Active
12	PV	Sindhu	F	2023-02-01	4	7	5	Employee	Active
13	Virat	Kohli	M	2022-06-15	4	7	5	Employee	Active
14	Naomi	Osaka	F	2017-10-23	5	9	5	Employee	Active
15	Usain	Bolt	M	2024-01-08	6	10	1	Employee	Active
16	Sania	Mirza	F	2021-03-22	7	10	1	Employee	Active
17	Novak	Djokovic	M	2017-05-29	1	12	2	Employee	Active
18	Mary	Kom	F	2020-02-17	9	13	1	Employee	Active
19	Zlatan	Ibrahimovic	M	2016-12-05	2	3	4	Employee	Resigned
20	Coco	Gauff	F	2025-07-14	2	2	4	Employee	Active

SALARY_COMPONENT

component_id	component_name	component_type	calc_method	value	is_taxable	is_active
1	House Rent Allowance (HRA)	Allowance	Percent	40.00	true	true
2	Dearness Allowance (DA)	Allowance	Percent	12.00	true	true
3	Special Allowance	Allowance	Percent	10.00	true	true
4	Conveyance Allowance	Allowance	Fixed	1600.00	false	true
5	Medical Allowance	Allowance	Fixed	1250.00	false	true
6	Internet Allowance	Allowance	Fixed	1000.00	false	true
7	Night Shift Allowance	Allowance	Fixed	2000.00	true	false
8	Provident Fund (PF)	Deduction	Percent	12.00	false	true
9	Professional Tax	Deduction	Fixed	200.00	false	true
10	Income Tax (TDS)	Deduction	Percent	5.00	false	true
11	Group Health Insurance	Deduction	Fixed	500.00	false	true
12	Labour Welfare Fund	Deduction	Fixed	25.00	false	false

LEAVE_REQUEST

leave_id	emp_id	leave_type	start_date	end_date	status	applied_on	approved_by
1	6	Casual	2026-08-10	2026-08-11	Approved	2026-08-03	4
2	7	Sick	2026-08-19	2026-08-20	Approved	2026-08-19	4
3	12	Earned	2026-08-24	2026-08-28	Approved	2026-08-05	5
4	13	Unpaid	2026-09-07	2026-09-09	Approved	2026-08-31	5
5	8	Casual	2026-09-14	2026-09-14	Rejected	2026-09-10	4
6	10	Sick	2026-09-21	2026-09-23	Approved	2026-09-18	3
7	14	Earned	2026-09-28	2026-09-30	Approved	2026-09-12	5
8	16	Unpaid	2026-09-16	2026-09-17	Approved	2026-09-08	1
9	9	Casual	2026-10-05	2026-10-06	Approved	2026-09-29	4
10	20	Sick	2026-10-15	2026-10-16	Pending	2026-10-09	NULL

leave_id	emp_id	leave_type	start_date	end_date	status	applied_on	approved_by
11	11	Earned	2026-10-26	2026-10-30	Pending	2026-10-07	NULL
12	17	Casual	2026-10-19	2026-10-19	Pending	2026-10-08	NULL
13	6	Earned	2026-11-09	2026-11-13	Pending	2026-10-09	NULL

ATTENDANCE (Virat Kohli, 1–14 September 2026)

attendance_id	emp_id	att_date	check_in	check_out	status
433	13	2026-09-01	09:20:00	18:37:00	Present
452	13	2026-09-02	09:21:00	18:38:00	Present
471	13	2026-09-03	09:22:00	18:39:00	Present
490	13	2026-09-04	09:23:00	18:40:00	Present
509	13	2026-09-07	NULL	NULL	Leave
528	13	2026-09-08	NULL	NULL	Leave
547	13	2026-09-09	NULL	NULL	Leave
566	13	2026-09-10	09:29:00	18:46:00	Present
585	13	2026-09-11	09:30:00	18:47:00	Present
604	13	2026-09-14	09:33:00	18:00:00	Present

Virat Kohli's approved unpaid leave on 7–9 September appears as status `Leave` – it was written by trigger `trg_leave_approved`.

PAYROLL (September 2026, first 12 employees)

payroll_id	emp_id	pay_month	pay_year	working_days	lop_days	basic_salary	gross_salary	total_deductions	net_salary	status
21	1	9	2026	22	1.0	150000.00	235804.55	25040.91	210763.64	Generated
22	2	9	2026	22	1.5	85000.00	132161.38	14164.78	117996.60	Generated
23	3	9	2026	22	0.0	52000.00	88090.00	9540.00	78550.00	Generated
24	4	9	2026	22	1.0	95000.00	150754.55	16115.91	134638.64	Generated
25	5	9	2026	22	1.0	80000.00	127559.10	13681.82	113877.28	Generated
26	6	9	2026	22	1.5	65000.00	101970.45	10996.59	90973.86	Generated
27	7	9	2026	22	1.0	45000.00	73436.38	8002.28	65434.10	Generated
28	8	9	2026	22	0.5	45000.00	75093.18	8176.13	66917.05	Generated
29	9	9	2026	22	0.5	45000.00	75093.18	8176.13	66917.05	Generated
30	10	9	2026	22	1.0	40000.00	65704.55	7190.91	58513.64	Generated
31	11	9	2026	22	1.5	40000.00	64231.82	7036.37	57195.45	Generated
32	12	9	2026	22	1.5	30000.00	49136.38	5452.28	43684.10	Generated

PAYROLL_DETAIL (LeBron James, September 2026)

payroll_id	component_id	component_name	amount
26	1	House Rent Allowance (HRA)	24227.27
26	2	Dearness Allowance (DA)	7268.18
26	3	Special Allowance	6056.82
26	4	Conveyance Allowance	1600.00
26	5	Medical Allowance	1250.00
26	6	Internet Allowance	1000.00
26	8	Provident Fund (PF)	7268.18
26	9	Professional Tax	200.00
26	10	Income Tax (TDS)	3028.41
26	11	Group Health Insurance	500.00

6.4 DML Demonstration – INSERT, UPDATE and DELETE

Each statement below was executed on the populated database. Successful statements show the returned rows; rejected statements show the error PostgreSQL raised, which proves the constraint is enforced by the database itself.

1. INSERT a new employee

```
INSERT INTO employee (first_name, last_name, email, phone, gender, dob, date_of_joining,
                      pan_no, bank_account_no, ifsc_code, dept_id, desig_id, reports_to)
VALUES ('Kylian', 'Mbappe', 'kylian.mbappe@allcaps.example', '9840012321', 'M', '1998-12-20',
        '2026-10-01', 'AUKPM1221F', '501000123421', 'HDFC0001201', 2, 2, 4)
RETURNING emp_id, first_name, last_name, status, app_role;
```

emp_id	first_name	last_name	status	app_role
21	Kylian	Mbappe	Active	Employee

status and app_role take their DEFAULT values ('Active', 'Employee').

2. INSERT rejected by CHECK constraint (invalid PAN format)

```
INSERT INTO employee (first_name, last_name, email, phone, gender, dob, date_of_joining,
                      pan_no, bank_account_no, ifsc_code, dept_id, desig_id)
VALUES ('Test', 'User', 'test.user@allcaps.example', '9840012399', 'M', '1999-01-01',
        '2026-10-01', 'BADPAN', '501000999999', 'HDFC0001201', 2, 2);
```

ERROR: new row for relation "employee" violates check constraint "employee_pan_no_check"

The PAN must match ^[A-Z]{5}[0-9]{4}[A-Z]$.

3. INSERT rejected by UNIQUE constraint (attendance already recorded)

```
INSERT INTO attendance (emp_id, att_date, status)
VALUES (6, '2026-09-01', 'Present');
```

ERROR: duplicate key value violates unique constraint "uq_att_emp_day"

UNIQUE (emp_id, att_date) allows one row per employee per day.

4. INSERT rejected by FOREIGN KEY (department does not exist)

```
INSERT INTO employee (first_name, last_name, email, phone, gender, dob, date_of_joining,
                      pan_no, bank_account_no, ifsc_code, dept_id, desig_id)
VALUES ('Test', 'User', 'test.user@allcaps.example', '9840012399', 'M', '1999-01-01',
        '2026-10-01', 'AVTPU1299G', '501000999999', 'HDFC0001201', 99, 2);
```

ERROR: insert or update on table "employee" violates foreign key constraint "employee_dept_id_fkey"

dept_id 99 is not in DEPARTMENT.

5. UPDATE: annual salary revision of a designation

```
UPDATE designation
   SET basic_salary = basic_salary * 1.10
 WHERE title = 'Software Engineer'
RETURNING desig_id, title, basic_salary;
```

desig_id	title	basic_salary
2	Software Engineer	49500.00

Future payroll runs use the new basic; paid payslips keep their stored snapshot.

6. UPDATE: deactivate a salary component

```
UPDATE salary_component
   SET is_active = FALSE
 WHERE component_name = 'Internet Allowance'
RETURNING component_id, component_name, is_active;
```

component_id	component_name	is_active
6	Internet Allowance	false

Inactive components are skipped by generate_monthly_payroll.

7. DELETE: employee cancels a pending leave request

```
DELETE FROM leave_request
 WHERE leave_id = 13 AND status = 'Pending'
RETURNING leave_id, emp_id, leave_type, status;
```

leave_id	emp_id	leave_type	status
13	6	Earned	Pending

8. DELETE: remove the employee added above (no payroll yet)

```
DELETE FROM employee
 WHERE email = 'kylian.mbappe@allcaps.example'
RETURNING emp_id, first_name, last_name;
```

emp_id	first_name	last_name
21	Kylian	Mbappe

9. DELETE rejected by FOREIGN KEY (employee has payroll history)

```
DELETE FROM employee WHERE emp_id = 7;
```

ERROR: update or delete on table "employee" violates foreign key constraint "payroll_emp_id_fkey" on table "payroll"

payroll.emp_id references employee with ON DELETE RESTRICT (default NO ACTION), so salary history cannot be lost.

7. SQL Queries and Outputs

Nineteen queries answer real questions an HR or payroll team asks. Each is saved as a view in `sql/05_queries.sql` and can be run live from the SQL Queries page of the application (Figure 12). Outputs below are the actual results on the sample data (at most 10 rows shown).

#	Question answered	SQL concepts
Q1	Employee directory with department, designation and manager	INNER JOIN, LEFT SELF-JOIN
Q2	Head-count and average basic salary of every department (including empty ones)	LEFT JOIN, GROUP BY, COUNT / AVG
Q3	Employees whose basic salary is above the company average	Scalar subquery, Comparison
Q4	Department-wise salary cost for the latest payroll month	CTE (WITH), JOIN, SUM, GROUP BY
Q5	Top 5 earners by net salary in the latest month (with rank)	Window function RANK(), ORDER BY, LIMIT
Q6	Attendance summary of every employee for September 2026	Conditional aggregation (FILTER), GROUP BY
Q7	Employees who lost pay (LOP) in any month, with the amount lost	WHERE filter, Arithmetic expression, JOIN
Q8	Remaining 2026 leave balance of every active employee	User-defined FUNCTION in SELECT
Q9	Pending leave requests and the manager who must approve them	Self-join, Date arithmetic
Q10	Managers with at least 3 direct reports	Self-join, GROUP BY, HAVING
Q11	Itemised payslip lines of every employee for the latest month	4-table JOIN, M:N bridge table
Q12	Active employees who have not taken any approved leave in 2026	NOT EXISTS (correlated subquery)
Q13	Month-by-month payroll totals and payment status	GROUP BY multiple columns, SUM, CASE
Q14	Departments whose average net salary is above the company average (latest month)	HAVING with subquery, AVG
Q15	Age and tenure of every active employee (derived attributes)	Date functions AGE() / EXTRACT, Derived attributes
Q16	Total paid out per salary component in the latest month	JOIN, GROUP BY, ROLLUP-style totals with UNION ALL
Q17	Employees who were both absent and on approved leave in September	INTERSECT (set operation)
Q18	Everyone with leadership responsibility: department heads or people managers	UNION (set operation), IN subquery
Q19	Employees who checked in after 09:35 more than 10 times since August	Time comparison, GROUP BY, HAVING

Q1. Employee directory with department, designation and manager

Concepts: INNER JOIN, LEFT SELF-JOIN

```
SELECT e.emp_id,
       e.first_name || ' ' || e.last_name AS employee,
       d.dept_name,
       g.title AS designation,
       m.first_name || ' ' || m.last_name AS manager,
       e.status
  FROM employee e
  JOIN department  d ON d.dept_id  = e.dept_id
  JOIN designation g ON g.desig_id = e.desig_id
  LEFT JOIN employee m ON m.emp_id = e.reports_to
 ORDER BY e.emp_id;
```

Output – 20 row(s), first 10 shown:

emp_id	employee	dept_name	designation	manager	status
1	Sachin Tendulkar	Administration	Director	NULL	Active
2	Daniel Dumile	Human Resources	HR Manager	Sachin Tendulkar	Active

emp_id	employee	dept_name	designation	manager	status
3	Jeffery Williams	Finance	Payroll Officer	Sachin Tendulkar	Active
4	Aubrey Graham	Engineering	Engineering Manager	Sachin Tendulkar	Active
5	Michael Jordan	Sales	Sales Manager	Sachin Tendulkar	Active
6	LeBron James	Engineering	Senior Software Engineer	Aubrey Graham	Active
7	Cristiano Ronaldo	Engineering	Software Engineer	Aubrey Graham	Active
8	Serena Williams	Engineering	Software Engineer	Aubrey Graham	Active
9	Lionel Messi	Quality Assurance	Software Engineer	Aubrey Graham	Active
10	Simone Biles	Finance	Accountant	Jeffery Williams	Active

Q2. Head-count and average basic salary of every department (including empty ones)

Concepts: LEFT JOIN, GROUP BY, COUNT / AVG

```
SELECT d.dept_name,
       d.location,
       COUNT(e.emp_id) AS active_employees,
       ROUND(AVG(g.basic_salary), 2) AS avg_basic_salary
  FROM department d
  LEFT JOIN employee e    ON e.dept_id = d.dept_id AND e.status = 'Active'
  LEFT JOIN designation g ON g.desig_id = e.desig_id
 GROUP BY d.dept_id, d.dept_name, d.location
 ORDER BY active_employees DESC, d.dept_name;
```

Output – 10 row(s):

dept_name	location	active_employees	avg_basic_salary
Engineering	Bengaluru	5	59000.00
Finance	Chennai	3	44000.00
Sales	Mumbai	3	46666.67
Human Resources	Chennai	2	61500.00
Administration	Vellore	1	150000.00
Customer Support	Hyderabad	1	28000.00
Legal	New Delhi	1	70000.00
Marketing	Mumbai	1	42000.00
Operations	Pune	1	28000.00
Quality Assurance	Bengaluru	1	45000.00

Q3. Employees whose basic salary is above the company average

Concepts: Scalar subquery, Comparison

```
SELECT e.emp_id,
       e.first_name || ' ' || e.last_name AS employee,
       g.title,
       g.basic_salary
  FROM employee e
  JOIN designation g ON g.desig_id = e.desig_id
 WHERE e.status = 'Active'
   AND g.basic_salary > (SELECT AVG(g2.basic_salary)
                           FROM employee e2
                           JOIN designation g2 ON g2.desig_id = e2.desig_id
                          WHERE e2.status = 'Active')
 ORDER BY g.basic_salary DESC;
```

Output – 6 row(s):

emp_id	employee	title	basic_salary
1	Sachin Tendulkar	Director	150000.00
4	Aubrey Graham	Engineering Manager	95000.00
2	Daniel Dumile	HR Manager	85000.00
5	Michael Jordan	Sales Manager	80000.00
18	Mary Kom	Legal Counsel	70000.00
6	LeBron James	Senior Software Engineer	65000.00

Q4. Department-wise salary cost for the latest payroll month

Concepts: CTE (WITH), JOIN, SUM, GROUP BY

```
WITH latest AS (
    SELECT pay_month, pay_year
      FROM payroll
     ORDER BY pay_year DESC, pay_month DESC
     LIMIT 1
)
SELECT d.dept_name,
       COUNT(*)              AS employees_paid,
       SUM(p.gross_salary)   AS total_gross,
       SUM(p.total_deductions) AS total_deductions,
       SUM(p.net_salary)     AS total_net
  FROM payroll p
  JOIN latest l     ON l.pay_month = p.pay_month AND l.pay_year = p.pay_year
  JOIN employee e   ON e.emp_id = p.emp_id
  JOIN department d ON d.dept_id = e.dept_id
 GROUP BY d.dept_name
 ORDER BY total_net DESC;
```

Output – 10 row(s):

dept_name	employees_paid	total_gross	total_deductions	total_net
Engineering	5	473034.11	51119.32	421914.79
Administration	1	235804.55	25040.91	210763.64
Sales	3	222518.21	24238.64	198279.57
Finance	3	218026.37	23767.28	194259.09
Human Resources	2	193374.11	20884.32	172489.79
Legal	1	114672.73	12329.54	102343.19
Quality Assurance	1	75093.18	8176.13	66917.05
Marketing	1	71890.00	7840.00	64050.00
Operations	1	49210.00	5460.00	43750.00
Customer Support	1	43024.55	4810.91	38213.64

Q5. Top 5 earners by net salary in the latest month (with rank)

Concepts: Window function RANK(), ORDER BY, LIMIT

```
WITH latest AS (
    SELECT pay_month, pay_year
      FROM payroll
     ORDER BY pay_year DESC, pay_month DESC
     LIMIT 1
)
SELECT RANK() OVER (ORDER BY p.net_salary DESC) AS rank,
       e.first_name || ' ' || e.last_name AS employee,
```

```
       g.title,
       p.net_salary
  FROM payroll p
  JOIN latest l      ON l.pay_month = p.pay_month AND l.pay_year = p.pay_year
  JOIN employee e    ON e.emp_id = p.emp_id
  JOIN designation g ON g.desig_id = e.desig_id
 ORDER BY p.net_salary DESC
 LIMIT 5;
```

Output – 5 row(s):

rank	employee	title	net_salary
1	Sachin Tendulkar	Director	210763.64
2	Aubrey Graham	Engineering Manager	134638.64
3	Daniel Dumile	HR Manager	117996.60
4	Michael Jordan	Sales Manager	113877.28
5	Mary Kom	Legal Counsel	102343.19

Q6. Attendance summary of every employee for September 2026

Concepts: Conditional aggregation (FILTER), GROUP BY

```
SELECT e.emp_id,
       e.first_name || ' ' || e.last_name AS employee,
       COUNT(*) FILTER (WHERE a.status = 'Present')  AS present,
       COUNT(*) FILTER (WHERE a.status = 'Half-Day') AS half_days,
       COUNT(*) FILTER (WHERE a.status = 'Absent')   AS absent,
       COUNT(*) FILTER (WHERE a.status = 'Leave')    AS on_leave
  FROM employee e
  JOIN attendance a ON a.emp_id = e.emp_id
 WHERE a.att_date BETWEEN DATE '2026-09-01' AND DATE '2026-09-30'
 GROUP BY e.emp_id, e.first_name, e.last_name
 ORDER BY absent DESC, half_days DESC, e.emp_id;
```

Output – 19 row(s), first 10 shown:

emp_id	employee	present	half_days	absent	on_leave
2	Daniel Dumile	20	1	1	0
6	LeBron James	20	1	1	0
11	Rafael Nadal	20	1	1	0
12	PV Sindhu	20	1	1	0
17	Novak Djokovic	20	1	1	0
20	Coco Gauff	20	1	1	0
1	Sachin Tendulkar	21	0	1	0
4	Aubrey Graham	21	0	1	0
5	Michael Jordan	21	0	1	0
7	Cristiano Ronaldo	21	0	1	0

Q7. Employees who lost pay (LOP) in any month, with the amount lost

Concepts: WHERE filter, Arithmetic expression, JOIN

```
SELECT p.pay_month,
       p.pay_year,
       e.first_name || ' ' || e.last_name AS employee,
       p.lop_days,
       ROUND(p.basic_salary / p.working_days * p.lop_days, 2) AS basic_lost
  FROM payroll p
```

```
  JOIN employee e ON e.emp_id = p.emp_id
 WHERE p.lop_days > 0
 ORDER BY p.pay_year, p.pay_month, basic_lost DESC;
```

Output – 32 row(s), first 10 shown:

pay_month	pay_year	employee	lop_days	basic_lost
8	2026	Michael Jordan	1.5	5714.29
8	2026	Mary Kom	1.5	5000.00
8	2026	Sachin Tendulkar	0.5	3571.43
8	2026	Coco Gauff	1.5	3214.29
8	2026	Serena Williams	1.5	3214.29
8	2026	Zlatan Ibrahimovic	1.0	3095.24
8	2026	Jeffery Williams	1.0	2476.19
8	2026	Lionel Messi	1.0	2142.86
8	2026	Naomi Osaka	1.0	2000.00
8	2026	Usain Bolt	1.5	2000.00

Q8. Remaining 2026 leave balance of every active employee

Concepts: User-defined FUNCTION in SELECT

```
SELECT e.emp_id,
       e.first_name || ' ' || e.last_name AS employee,
       g.annual_leave_quota AS quota,
       fn_leave_balance(e.emp_id, 2026) AS balance
  FROM employee e
  JOIN designation g ON g.desig_id = e.desig_id
 WHERE e.status = 'Active'
 ORDER BY balance, e.emp_id;
```

Output – 19 row(s), first 10 shown:

emp_id	employee	quota	balance
12	PV Sindhu	18	13
10	Simone Biles	20	17
14	Naomi Osaka	20	17
7	Cristiano Ronaldo	20	18
9	Lionel Messi	20	18
13	Virat Kohli	18	18
15	Usain Bolt	18	18
16	Sania Mirza	18	18
6	LeBron James	22	20
8	Serena Williams	20	20

Q9. Pending leave requests and the manager who must approve them

Concepts: Self-join, Date arithmetic

```
SELECT lr.leave_id,
       e.first_name || ' ' || e.last_name AS employee,
       lr.leave_type,
       lr.start_date,
       lr.end_date,
       (lr.end_date - lr.start_date + 1) AS calendar_days,
```

```
       COALESCE(m.first_name || ' ' || m.last_name, '(Admin)') AS approver
  FROM leave_request lr
  JOIN employee e      ON e.emp_id = lr.emp_id
  LEFT JOIN employee m ON m.emp_id = e.reports_to
 WHERE lr.status = 'Pending'
 ORDER BY lr.start_date;
```

Output – 4 row(s):

leave_id	employee	leave_type	start_date	end_date	calendar_days	approver
10	Coco Gauff	Sick	2026-10-15	2026-10-16	2	Aubrey Graham
12	Novak Djokovic	Casual	2026-10-19	2026-10-19	1	Daniel Dumile
11	Rafael Nadal	Earned	2026-10-26	2026-10-30	5	Jeffery Williams
13	LeBron James	Earned	2026-11-09	2026-11-13	5	Aubrey Graham

Q10. Managers with at least 3 direct reports

Concepts: Self-join, GROUP BY, HAVING

```
SELECT m.emp_id,
       m.first_name || ' ' || m.last_name AS manager,
       COUNT(e.emp_id) AS direct_reports
  FROM employee m
  JOIN employee e ON e.reports_to = m.emp_id AND e.status = 'Active'
 GROUP BY m.emp_id, m.first_name, m.last_name
HAVING COUNT(e.emp_id) >= 3
 ORDER BY direct_reports DESC;
```

Output – 3 row(s):

emp_id	manager	direct_reports
1	Sachin Tendulkar	7
4	Aubrey Graham	5
5	Michael Jordan	3

Q11. Itemised payslip lines of every employee for the latest month

Concepts: 4-table JOIN, M:N bridge table

```
WITH latest AS (
    SELECT pay_month, pay_year
      FROM payroll
     ORDER BY pay_year DESC, pay_month DESC
     LIMIT 1
)
SELECT e.emp_id,
       e.first_name || ' ' || e.last_name AS employee,
       c.component_type,
       c.component_name,
       pd.amount
  FROM payroll p
  JOIN latest l           ON l.pay_month = p.pay_month AND l.pay_year = p.pay_year
  JOIN employee e         ON e.emp_id = p.emp_id
  JOIN payroll_detail pd  ON pd.payroll_id = p.payroll_id
  JOIN salary_component c ON c.component_id = pd.component_id
 ORDER BY e.emp_id, c.component_type, pd.amount DESC;
```

Output – 190 row(s), first 10 shown:

emp_id	employee	component_type	component_name	amount
1	Sachin Tendulkar	Allowance	House Rent Allowance (HRA)	57272.73
1	Sachin Tendulkar	Allowance	Dearness Allowance (DA)	17181.82

emp_id	employee	component_type	component_name	amount
1	Sachin Tendulkar	Allowance	Special Allowance	14318.18
1	Sachin Tendulkar	Allowance	Conveyance Allowance	1600.00
1	Sachin Tendulkar	Allowance	Medical Allowance	1250.00
1	Sachin Tendulkar	Allowance	Internet Allowance	1000.00
1	Sachin Tendulkar	Deduction	Provident Fund (PF)	17181.82
1	Sachin Tendulkar	Deduction	Income Tax (TDS)	7159.09
1	Sachin Tendulkar	Deduction	Group Health Insurance	500.00
1	Sachin Tendulkar	Deduction	Professional Tax	200.00

Q12. Active employees who have not taken any approved leave in 2026

Concepts: NOT EXISTS (correlated subquery)

```
SELECT e.emp_id,
       e.first_name || ' ' || e.last_name AS employee,
       d.dept_name
  FROM employee e
  JOIN department d ON d.dept_id = e.dept_id
 WHERE e.status = 'Active'
   AND NOT EXISTS (SELECT 1
                     FROM leave_request lr
                    WHERE lr.emp_id = e.emp_id
                      AND lr.status = 'Approved'
                      AND EXTRACT(YEAR FROM lr.start_date) = 2026)
 ORDER BY e.emp_id;
```

Output – 11 row(s), first 10 shown:

emp_id	employee	dept_name
1	Sachin Tendulkar	Administration
2	Daniel Dumile	Human Resources
3	Jeffery Williams	Finance
4	Aubrey Graham	Engineering
5	Michael Jordan	Sales
8	Serena Williams	Engineering
11	Rafael Nadal	Finance
15	Usain Bolt	Operations
17	Novak Djokovic	Human Resources
18	Mary Kom	Legal

Q13. Month-by-month payroll totals and payment status

Concepts: GROUP BY multiple columns, SUM, CASE

```
SELECT pay_year,
       pay_month,
       COUNT(*) AS payslips,
       SUM(gross_salary) AS total_gross,
       SUM(net_salary)   AS total_net,
       CASE WHEN BOOL_AND(status = 'Paid') THEN 'All paid'
            ELSE COUNT(*) FILTER (WHERE status <> 'Paid') || ' unpaid' END AS payment_status
  FROM payroll
 GROUP BY pay_year, pay_month
 ORDER BY pay_year, pay_month;
```

Output – 2 row(s):

pay_year	pay_month	payslips	total_gross	total_net	payment_status
2026	8	20	1825944.25	1628413.03	All paid
2026	9	19	1696647.81	1512980.76	19 unpaid

Q14. Departments whose average net salary is above the company average (latest month)

Concepts: HAVING with subquery, AVG

```
WITH latest AS (
    SELECT pay_month, pay_year
      FROM payroll
     ORDER BY pay_year DESC, pay_month DESC
     LIMIT 1
)
SELECT d.dept_name,
       ROUND(AVG(p.net_salary), 2) AS avg_net_salary
  FROM payroll p
  JOIN latest l     ON l.pay_month = p.pay_month AND l.pay_year = p.pay_year
  JOIN employee e   ON e.emp_id = p.emp_id
  JOIN department d ON d.dept_id = e.dept_id
 GROUP BY d.dept_name
HAVING AVG(p.net_salary) > (SELECT AVG(p2.net_salary)
                            FROM payroll p2
                            JOIN latest l2 ON l2.pay_month = p2.pay_month AND l2.pay_year =
p2.pay_year)
 ORDER BY avg_net_salary DESC;
```

Output – 4 row(s):

dept_name	avg_net_salary
Administration	210763.64
Legal	102343.19
Human Resources	86244.90
Engineering	84382.96

Q15. Age and tenure of every active employee (derived attributes)

Concepts: Date functions AGE() / EXTRACT, Derived attributes

```
SELECT e.emp_id,
       e.first_name || ' ' || e.last_name AS employee,
       EXTRACT(YEAR FROM AGE(CURRENT_DATE, e.dob))::INT AS age,
       e.date_of_joining,
       EXTRACT(YEAR FROM AGE(CURRENT_DATE, e.date_of_joining))::INT AS years_of_service
  FROM employee e
 WHERE e.status = 'Active'
 ORDER BY years_of_service DESC, e.emp_id;
```

Output – 19 row(s), first 10 shown:

emp_id	employee	age	date_of_joining	years_of_service
1	Sachin Tendulkar	51	2012-06-01	14
5	Michael Jordan	43	2013-08-05	13
4	Aubrey Graham	41	2014-01-20	12
2	Daniel Dumile	41	2015-03-16	11
3	Jeffery Williams	38	2016-07-11	10
17	Novak Djokovic	34	2017-05-29	9

emp_id	employee	age	date_of_joining	years_of_service
11	Rafael Nadal	32	2018-09-03	8
14	Naomi Osaka	33	2017-10-23	8
6	LeBron James	30	2019-07-01	7
10	Simone Biles	30	2019-04-08	7

Q16. Total paid out per salary component in the latest month

Concepts: JOIN, GROUP BY, ROLLUP-style totals with UNION ALL

```
WITH latest AS (
    SELECT pay_month, pay_year
      FROM payroll
     ORDER BY pay_year DESC, pay_month DESC
     LIMIT 1
),
lines AS (
    SELECT c.component_type, c.component_name, pd.amount
      FROM payroll p
      JOIN latest l           ON l.pay_month = p.pay_month AND l.pay_year = p.pay_year
      JOIN payroll_detail pd  ON pd.payroll_id = p.payroll_id
      JOIN salary_component c ON c.component_id = pd.component_id
)
SELECT component_type, component_name, SUM(amount) AS total
  FROM lines
 GROUP BY component_type, component_name
UNION ALL
SELECT component_type, '— TOTAL ' || UPPER(component_type) || 'S', SUM(amount)
  FROM lines
 GROUP BY component_type
 ORDER BY component_type, total;
```

Output – 12 row(s), first 10 shown:

component_type	component_name	total
Allowance	Internet Allowance	19000.00
Allowance	Medical Allowance	23750.00
Allowance	Conveyance Allowance	30400.00
Allowance	Special Allowance	100215.92
Allowance	Dearness Allowance (DA)	120259.11
Allowance	House Rent Allowance (HRA)	400863.67
Allowance	— TOTAL ALLOWANCES	694488.70
Deduction	Professional Tax	3800.00
Deduction	Group Health Insurance	9500.00
Deduction	Income Tax (TDS)	50107.94

Q17. Employees who were both absent and on approved leave in September

Concepts: INTERSECT (set operation)

```
SELECT e.emp_id, e.first_name || ' ' || e.last_name AS employee
  FROM employee e
  JOIN attendance a ON a.emp_id = e.emp_id
 WHERE a.status IN ('Absent', 'Half-Day')
   AND a.att_date BETWEEN DATE '2026-09-01' AND DATE '2026-09-30'
INTERSECT
SELECT e.emp_id, e.first_name || ' ' || e.last_name
  FROM employee e
  JOIN leave_request lr ON lr.emp_id = e.emp_id
```

```
 WHERE lr.status = 'Approved'
   AND lr.start_date <= DATE '2026-09-30' AND lr.end_date >= DATE '2026-09-01'
 ORDER BY emp_id;
```

Output – 2 row(s):

emp_id	employee
10	Simone Biles
16	Sania Mirza

Q18. Everyone with leadership responsibility: department heads or people managers

Concepts: UNION (set operation), IN subquery

```
SELECT e.emp_id, e.first_name || ' ' || e.last_name AS employee, 'Department head' AS
responsibility
  FROM employee e
 WHERE e.emp_id IN (SELECT head_emp_id FROM department WHERE head_emp_id IS NOT NULL)
UNION
SELECT m.emp_id, m.first_name || ' ' || m.last_name, 'People manager'
  FROM employee m
 WHERE EXISTS (SELECT 1 FROM employee e WHERE e.reports_to = m.emp_id AND e.status = 'Active')
 ORDER BY emp_id, responsibility;
```

Output – 11 row(s), first 10 shown:

emp_id	employee	responsibility
1	Sachin Tendulkar	Department head
1	Sachin Tendulkar	People manager
2	Daniel Dumile	Department head
2	Daniel Dumile	People manager
3	Jeffery Williams	Department head
3	Jeffery Williams	People manager
4	Aubrey Graham	Department head
4	Aubrey Graham	People manager
5	Michael Jordan	Department head
5	Michael Jordan	People manager

Q19. Employees who checked in after 09:35 more than 10 times since August

Concepts: Time comparison, GROUP BY, HAVING

```
SELECT e.emp_id,
       e.first_name || ' ' || e.last_name AS employee,
       COUNT(*) AS late_days,
       MAX(a.check_in) AS latest_check_in
  FROM attendance a
  JOIN employee e ON e.emp_id = a.emp_id
 WHERE a.check_in > TIME '09:35'
 GROUP BY e.emp_id, e.first_name, e.last_name
HAVING COUNT(*) > 10
 ORDER BY late_days DESC;
```

Output – 10 row(s):

emp_id	employee	late_days	latest_check_in
11	Rafael Nadal	14	09:44:00

emp_id	employee	late_days	latest_check_in
12	PV Sindhu	14	09:44:00
13	Virat Kohli	14	09:44:00
18	Mary Kom	13	09:44:00
5	Michael Jordan	13	09:44:00
17	Novak Djokovic	12	09:44:00
6	LeBron James	11	09:44:00
20	Coco Gauff	11	09:44:00
4	Aubrey Graham	11	09:43:00
7	Cristiano Ronaldo	11	09:44:00

8. Procedures, Functions, Cursor and Triggers

All server-side logic is written in PL/pgSQL, PostgreSQL's procedural language (the equivalent of Oracle PL/SQL). It lives in `sql/02_functions_procedures.sql` and `sql/03_triggers.sql`.

8.1 Overview

Type	Name	Business purpose	Used by
Function	fn_working_days(month, year)	Working days (Mon–Fri) in a month	generate_monthly_payroll
Function	fn_component_amount(component, basic)	Amount of a fixed or % salary component	generate_monthly_payroll
Function	fn_lop_days(emp, month, year)	Loss-of-pay days: absent + ½ half-day + unpaid leave	generate_monthly_payroll, Q7
Function	fn_leave_balance(emp, year)	Remaining paid leave for the year	process_leave, Q8, Leave page
Procedure + cursor	generate_monthly_payroll(month, year)	Creates every active employee's payroll and payslip lines	Payroll Run page
Procedure	process_leave(leave, approver, decision)	Validates and approves/rejects a leave request	Leave approvals page
Procedure + cursor	pay_monthly_payroll(month, year, mode)	Marks the month's payroll Paid with a transaction reference	Payroll Run page
Trigger	trg_leave_approved	Approved leave → 'Leave' rows in ATTENDANCE	leave_request UPDATE
Trigger	trg_lock_paid_payroll	Paid payroll cannot be changed or deleted	payroll UPDATE/DELETE
Trigger	trg_validate_manager	Manager must be active; no reporting cycles	employee INSERT/UPDATE
Trigger	trg_employee_exit	On resignation: reassign team, clear headship, reject pending leave	employee UPDATE

8.2 Functions

Function 1 – fn_working_days

```
-- ---------------------------------------------------------------------
-- FUNCTION 1: fn_working_days(month, year)
--   Number of working days (Monday-Friday) in a month.
--   Used by: generate_monthly_payroll
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION fn_working_days(p_month INT, p_year INT)
RETURNS INT
LANGUAGE plpgsql IMMUTABLE
AS $$
DECLARE
    v_first DATE := make_date(p_year, p_month, 1);
    v_last  DATE := (make_date(p_year, p_month, 1) + INTERVAL '1 month' - INTERVAL '1 day')::DATE;
BEGIN
    RETURN (SELECT COUNT(*)
              FROM generate_series(v_first, v_last, INTERVAL '1 day') AS d
             WHERE EXTRACT(ISODOW FROM d) < 6);
END;
$$;
```

Example:

aug_2026	sep_2026	oct_2026
21	22	22

Function 2 – fn_component_amount

```
-- ---------------------------------------------------------------------
-- FUNCTION 2: fn_component_amount(component_id, basic)
--   Amount of one salary component for a given (earned) basic salary.
--   Fixed components return their value; Percent components return
--   value% of basic.
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION fn_component_amount(p_component_id INT, p_basic NUMERIC)
RETURNS NUMERIC
LANGUAGE plpgsql STABLE
AS $$
DECLARE
    v_method salary_component.calc_method%TYPE;
    v_value  salary_component.value%TYPE;
BEGIN
    SELECT calc_method, value INTO v_method, v_value
      FROM salary_component
     WHERE component_id = p_component_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Salary component % does not exist', p_component_id;
    END IF;

    IF v_method = 'Fixed' THEN
        RETURN v_value;
    END IF;
    RETURN ROUND(p_basic * v_value / 100, 2);
END;
$$;
```

Example – every active component for a basic salary of ₹45,000:

component_name	calc_method	value	amount_for_basic_45000
House Rent Allowance (HRA)	Percent	40.00	18000.00
Dearness Allowance (DA)	Percent	12.00	5400.00
Special Allowance	Percent	10.00	4500.00
Conveyance Allowance	Fixed	1600.00	1600.00
Medical Allowance	Fixed	1250.00	1250.00
Internet Allowance	Fixed	1000.00	1000.00
Provident Fund (PF)	Percent	12.00	5400.00
Professional Tax	Fixed	200.00	200.00
Income Tax (TDS)	Percent	5.00	2250.00
Group Health Insurance	Fixed	500.00	500.00

Function 3 – fn_lop_days

```
-- ---------------------------------------------------------------------
-- FUNCTION 3: fn_lop_days(emp_id, month, year)
--   Loss-of-pay days = Absent days + 0.5 x Half-Day days
--                    + weekdays covered by APPROVED UNPAID leave.
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION fn_lop_days(p_emp_id INT, p_month INT, p_year INT)
RETURNS NUMERIC
LANGUAGE plpgsql STABLE
AS $$
DECLARE
    v_first   DATE := make_date(p_year, p_month, 1);
    v_last    DATE := (make_date(p_year, p_month, 1) + INTERVAL '1 month' - INTERVAL '1
day')::DATE;
    v_absent  NUMERIC;
    v_unpaid  NUMERIC;
BEGIN
    SELECT COALESCE(SUM(CASE status WHEN 'Absent' THEN 1
                                    WHEN 'Half-Day' THEN 0.5
                                    ELSE 0 END), 0)
```

```
      INTO v_absent
      FROM attendance
     WHERE emp_id = p_emp_id
       AND att_date BETWEEN v_first AND v_last;

    SELECT COUNT(*)
      INTO v_unpaid
      FROM leave_request lr
      CROSS JOIN LATERAL generate_series(GREATEST(lr.start_date, v_first),
                                         LEAST(lr.end_date, v_last),
                                         INTERVAL '1 day') AS d
     WHERE lr.emp_id = p_emp_id
       AND lr.leave_type = 'Unpaid'
       AND lr.status = 'Approved'
       AND EXTRACT(ISODOW FROM d) < 6;

    RETURN v_absent + v_unpaid;
END;
$$;
```

Example – September 2026:

emp_id	employee	lop_sep_2026
2	Daniel Dumile	1.5
10	Simone Biles	1
13	Virat Kohli	3
16	Sania Mirza	3

Virat Kohli and Sania Mirza each had 2–3 days of approved **unpaid** leave in September; Daniel Dumile's 1.5 days come from one absence and one half day.

Function 4 – fn_leave_balance

```
-- ---------------------------------------------------------------------
-- FUNCTION 4: fn_leave_balance(emp_id, year)
--   Remaining paid leave = designation quota - approved paid leave
--   (Casual/Sick/Earned) weekdays taken in that year.
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION fn_leave_balance(p_emp_id INT, p_year INT)
RETURNS INT
LANGUAGE plpgsql STABLE
AS $$
DECLARE
    v_quota INT;
    v_taken INT;
BEGIN
    SELECT d.annual_leave_quota INTO v_quota
      FROM employee e JOIN designation d ON d.desig_id = e.desig_id
     WHERE e.emp_id = p_emp_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Employee % does not exist', p_emp_id;
    END IF;

    SELECT COUNT(*) INTO v_taken
      FROM leave_request lr
      CROSS JOIN LATERAL generate_series(lr.start_date, lr.end_date, INTERVAL '1 day') AS d
     WHERE lr.emp_id = p_emp_id
       AND lr.status = 'Approved'
       AND lr.leave_type <> 'Unpaid'
       AND EXTRACT(YEAR FROM d) = p_year
       AND EXTRACT(ISODOW FROM d) < 6;

    RETURN v_quota - v_taken;
END;
$$;
```

Example:

emp_id	employee	quota	balance
6	LeBron James	22	20
7	Cristiano Ronaldo	20	18
12	PV Sindhu	18	13
14	Naomi Osaka	20	17

8.3 Procedures and Cursors

Procedure 1 – generate_monthly_payroll (explicit cursor)

```
-- ------------------------------------------------------------------------
-- PROCEDURE 1: generate_monthly_payroll(month, year, INOUT generated)
--   Uses an explicit CURSOR over all active employees. For each one:
--     LOP days -> earned basic -> every active salary component
--     -> PAYROLL row + PAYROLL_DETAIL line items.
--   Employees who already have a payroll for that month are skipped,
--   so the procedure can safely be run again.
-- ------------------------------------------------------------------------
CREATE OR REPLACE PROCEDURE generate_monthly_payroll(
    p_month INT,
    p_year INT,
    INOUT p_generated INT DEFAULT 0)
LANGUAGE plpgsql
AS $$
DECLARE
    v_month_end   DATE := (make_date(p_year, p_month, 1) + INTERVAL '1 month' - INTERVAL '1
day')::DATE;
    v_work_days   INT  := fn_working_days(p_month, p_year);

    cur_emp CURSOR FOR
        SELECT e.emp_id, d.basic_salary
          FROM employee e
          JOIN designation d ON d.desig_id = e.desig_id
         WHERE e.status IN ('Active', 'On Leave')
           AND e.date_of_joining <= v_month_end
         ORDER BY e.emp_id;

    rec           RECORD;
    comp          RECORD;
    v_lop         NUMERIC;
    v_earned      NUMERIC;
    v_amount      NUMERIC;
    v_allow       NUMERIC;
    v_deduct      NUMERIC;
    v_payroll_id  INT;
BEGIN
    p_generated := 0;
    OPEN cur_emp;
    LOOP
        FETCH cur_emp INTO rec;
        EXIT WHEN NOT FOUND;

        CONTINUE WHEN EXISTS (SELECT 1 FROM payroll
                              WHERE emp_id = rec.emp_id
                                AND pay_month = p_month AND pay_year = p_year);

        v_lop    := LEAST(fn_lop_days(rec.emp_id, p_month, p_year), v_work_days);
        v_earned := ROUND(rec.basic_salary - rec.basic_salary / v_work_days * v_lop, 2);
        v_allow  := 0;
        v_deduct := 0;

        INSERT INTO payroll (emp_id, pay_month, pay_year, working_days, lop_days, basic_salary)
        VALUES (rec.emp_id, p_month, p_year, v_work_days, v_lop, rec.basic_salary)
        RETURNING payroll_id INTO v_payroll_id;

        FOR comp IN SELECT component_id, component_type
                      FROM salary_component
                     WHERE is_active
```

```
                    ORDER BY component_id
        LOOP
            v_amount := fn_component_amount(comp.component_id, v_earned);
            INSERT INTO payroll_detail (payroll_id, component_id, amount)
            VALUES (v_payroll_id, comp.component_id, v_amount);

            IF comp.component_type = 'Allowance' THEN
                v_allow := v_allow + v_amount;
            ELSE
                v_deduct := v_deduct + v_amount;
            END IF;
        END LOOP;

        UPDATE payroll
           SET total_allowances = v_allow,
               total_deductions = v_deduct,
               gross_salary     = v_earned + v_allow,
               net_salary       = GREATEST(v_earned + v_allow - v_deduct, 0)
         WHERE payroll_id = v_payroll_id;

        p_generated := p_generated + 1;
    END LOOP;
    CLOSE cur_emp;

    RAISE NOTICE 'Payroll % / %: % employee(s) processed', p_month, p_year, p_generated;
END;
$$;
```

How the cursor is used: `cur_emp` is declared over all Active / On Leave employees who had joined by the end of the month. The procedure OPENs it, FETCHes one employee per loop iteration (EXIT WHEN NOT FOUND), computes LOP days, earned basic and every active component through the functions above, inserts one PAYROLL row and its PAYROLL_DETAIL lines, then CLOSEs the cursor. Employees who already have a payroll for that month are skipped with CONTINUE, so running it twice never creates duplicates.

Execution – October 2026, then the same call again:

p_generated
19

p_generated
0

emp_id	employee	working_days	lop_days	basic_salary	gross_salary	total_deductions	net_salary	status
1	Sachin Tendulkar	22	0.5	150000.00	240327.27	25620.46	214706.81	Generated
6	LeBron James	22	0.0	65000.00	108150.00	11750.00	96400.00	Generated
7	Cristiano Ronaldo	22	2.5	49500.00	73927.50	8158.75	65768.75	Generated
13	Virat Kohli	22	0.5	30000.00	50345.45	5684.09	44661.36	Generated
20	Coco Gauff	22	0.0	49500.00	83040.00	9115.00	73925.00	Generated

Cristiano Ronaldo has 2.5 LOP days: 0.5 from a half day plus 2 days of approved unpaid leave on 21–22 October (see 8.4). October was generated after the 10 % Software Engineer revision and the Internet Allowance deactivation from section 6.4, so Cristiano and Coco Gauff are paid the new basic of ₹49,500 and nobody receives the Internet Allowance.

Procedure 2 – process_leave

```
-- ------------------------------------------------------------------------
-- PROCEDURE 2: process_leave(leave_id, approver_id, decision)
--   Approves or rejects a pending leave request.
--   Rules: request must be Pending; approver must be the applicant's
--   manager or an Admin; paid leave cannot exceed the leave balance.
```

```
--   On approval, trigger trg_leave_approved updates ATTENDANCE.
-- ---------------------------------------------------------------------
CREATE OR REPLACE PROCEDURE process_leave(
    p_leave_id INT,
    p_approver_id INT,
    p_decision VARCHAR)
LANGUAGE plpgsql
AS $$
DECLARE
    v_req        leave_request%ROWTYPE;
    v_manager    INT;
    v_role       employee.app_role%TYPE;
    v_days       INT;
    v_balance    INT;
BEGIN
    IF p_decision NOT IN ('Approved', 'Rejected') THEN
        RAISE EXCEPTION 'Decision must be Approved or Rejected, got %', p_decision;
    END IF;

    SELECT * INTO v_req FROM leave_request WHERE leave_id = p_leave_id FOR UPDATE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Leave request % not found', p_leave_id;
    END IF;
    IF v_req.status <> 'Pending' THEN
        RAISE EXCEPTION 'Leave request % is already %', p_leave_id, v_req.status;
    END IF;

    SELECT reports_to INTO v_manager FROM employee WHERE emp_id = v_req.emp_id;
    SELECT app_role  INTO v_role    FROM employee WHERE emp_id = p_approver_id;
    IF p_approver_id = v_req.emp_id THEN
        RAISE EXCEPTION 'An employee cannot approve their own leave';
    END IF;
    IF v_role IS DISTINCT FROM 'Admin' AND p_approver_id IS DISTINCT FROM v_manager THEN
        RAISE EXCEPTION 'Employee % is not the manager of employee %', p_approver_id, v_req.emp_id;
    END IF;

    IF p_decision = 'Approved' AND v_req.leave_type <> 'Unpaid' THEN
        SELECT COUNT(*) INTO v_days
          FROM generate_series(v_req.start_date, v_req.end_date, INTERVAL '1 day') AS d
         WHERE EXTRACT(ISODOW FROM d) < 6;
        v_balance := fn_leave_balance(v_req.emp_id, EXTRACT(YEAR FROM v_req.start_date)::INT);
        IF v_days > v_balance THEN
            RAISE EXCEPTION 'Insufficient leave balance: requested %, available %', v_days,
v_balance;
        END IF;
    END IF;

    UPDATE leave_request
       SET status = p_decision,
           approved_by = p_approver_id
     WHERE leave_id = p_leave_id;
END;
$$;
```

Procedure 3 – pay_monthly_payroll (cursor FOR UPDATE … WHERE CURRENT OF)

```
-- ---------------------------------------------------------------------
-- PROCEDURE 3: pay_monthly_payroll(month, year, mode, INOUT paid)
--   Uses a CURSOR ... FOR UPDATE over all 'Generated' payroll rows of
--   the month, marks each one Paid and assigns a transaction reference.
-- ---------------------------------------------------------------------
CREATE OR REPLACE PROCEDURE pay_monthly_payroll(
    p_month INT,
    p_year INT,
    p_mode VARCHAR DEFAULT 'NEFT',
    INOUT p_paid INT DEFAULT 0)
LANGUAGE plpgsql
AS $$
DECLARE
    cur_pay CURSOR FOR
        SELECT payroll_id, emp_id
          FROM payroll
```

```
         WHERE pay_month = p_month AND pay_year = p_year AND status = 'Generated'
         ORDER BY emp_id
           FOR UPDATE;
    rec RECORD;
BEGIN
    p_paid := 0;
    OPEN cur_pay;
    LOOP
        FETCH cur_pay INTO rec;
        EXIT WHEN NOT FOUND;

        UPDATE payroll
           SET status          = 'Paid',
               payment_date    = (make_date(p_year, p_month, 1) + INTERVAL '1 month' - INTERVAL '1
day')::DATE,
               payment_mode    = p_mode,
               transaction_ref = 'TXN' || p_year || LPAD(p_month::TEXT, 2, '0') ||
LPAD(rec.emp_id::TEXT, 4, '0')
         WHERE CURRENT OF cur_pay;

        p_paid := p_paid + 1;
    END LOOP;
    CLOSE cur_pay;
END;
$$;
```

How the cursor is used: `cur_pay` selects the month's unpaid payroll rows FOR UPDATE, which locks them so two payroll officers cannot pay the same salaries at the same time. Each fetched row is updated through `WHERE CURRENT OF cur_pay`, the positioned update that changes exactly the row the cursor is on.

p_paid				
19				
emp_id	**status**	**payment_date**	**payment_mode**	**transaction_ref**
1	Paid	2026-10-31	NEFT	TXN2026100001
6	Paid	2026-10-31	NEFT	TXN2026100006
7	Paid	2026-10-31	NEFT	TXN2026100007

8.4 Triggers

Trigger 1 – trg_leave_approved

```
-- ------------------------------------------------------------------------
-- TRIGGER 1: trg_leave_approved   (AFTER UPDATE ON leave_request)
--   When a leave request becomes 'Approved', every weekday in the leave
--   range is written to ATTENDANCE with status 'Leave'. Existing rows for
--   those days are overwritten (UPSERT on the (emp_id, att_date) key).
-- ------------------------------------------------------------------------
CREATE OR REPLACE FUNCTION trgfn_leave_approved()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.status = 'Approved' AND OLD.status IS DISTINCT FROM 'Approved' THEN
        INSERT INTO attendance (emp_id, att_date, check_in, check_out, status)
        SELECT NEW.emp_id, d::DATE, NULL, NULL, 'Leave'
          FROM generate_series(NEW.start_date, NEW.end_date, INTERVAL '1 day') AS d
         WHERE EXTRACT(ISODOW FROM d) < 6
        ON CONFLICT (emp_id, att_date)
        DO UPDATE SET status = 'Leave', check_in = NULL, check_out = NULL;
    END IF;
    RETURN NEW;
END;
$$;
```

```
DROP TRIGGER IF EXISTS trg_leave_approved ON leave_request;
CREATE TRIGGER trg_leave_approved
AFTER UPDATE OF status ON leave_request
FOR EACH ROW
EXECUTE FUNCTION trgfn_leave_approved();
```

Coco Gauff's pending sick leave (15–16 October) is approved by her manager Aubrey Graham (employee 4). Attendance before and after:

Before: no attendance rows for 14–17 October (future dates).

RESULT: Statement completed successfully (no rows returned).

After:

att_date	status
2026-10-15	Leave
2026-10-16	Leave

leave_id	emp_id	leave_type	status	approved_by
10	20	Sick	Approved	4

The procedure process_leave also rejects invalid approvals:

Attempt	Database response
Michael Jordan (not his manager) approves Cristiano's leave	Employee 5 is not the manager of employee 7
Cristiano approves his own leave	An employee cannot approve their own leave
The same request is approved twice	Leave request 10 is already Approved
PV Sindhu requests 20 days of Earned leave	Insufficient leave balance: requested 20, available 13

Trigger 2 – trg_lock_paid_payroll

```
-- ------------------------------------------------------------------------
-- TRIGGER 2: trg_lock_paid_payroll   (BEFORE UPDATE OR DELETE ON payroll)
--   A payroll that has been Paid is a financial record: it can no longer
--   be changed or deleted.
-- ------------------------------------------------------------------------
CREATE OR REPLACE FUNCTION trgfn_lock_paid_payroll()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF OLD.status = 'Paid' THEN
        RAISE EXCEPTION 'Payroll % (employee %, %/%) is already Paid and cannot be %',
            OLD.payroll_id, OLD.emp_id, OLD.pay_month, OLD.pay_year,
            CASE TG_OP WHEN 'DELETE' THEN 'deleted' ELSE 'modified' END;
    END IF;
    IF TG_OP = 'DELETE' THEN
        RETURN OLD;
    END IF;
    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_lock_paid_payroll ON payroll;
CREATE TRIGGER trg_lock_paid_payroll
BEFORE UPDATE OR DELETE ON payroll
FOR EACH ROW
EXECUTE FUNCTION trgfn_lock_paid_payroll();
```

After October is paid, attempts to change or delete a payslip:

ERROR: Payroll 45 (employee 6, 10/2026) is already Paid and cannot be modified

ERROR: Payroll 45 (employee 6, 10/2026) is already Paid and cannot be deleted

Trigger 3 – trg_validate_manager

```
-- ------------------------------------------------------------------
-- TRIGGER 3: trg_validate_manager   (BEFORE INSERT OR UPDATE ON employee)
--   The reporting manager must exist, be Active, and must not report to
--   this employee (no two-person cycles).
-- ------------------------------------------------------------------
CREATE OR REPLACE FUNCTION trgfn_validate_manager()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_status   employee.status%TYPE;
    v_mgr_mgr  employee.reports_to%TYPE;
BEGIN
    IF NEW.reports_to IS NULL THEN
        RETURN NEW;
    END IF;

    SELECT status, reports_to INTO v_status, v_mgr_mgr
      FROM employee WHERE emp_id = NEW.reports_to;

    IF v_status IS DISTINCT FROM 'Active' THEN
        RAISE EXCEPTION 'Manager % is not an active employee', NEW.reports_to;
    END IF;
    IF v_mgr_mgr = NEW.emp_id THEN
        RAISE EXCEPTION 'Circular reporting: employee % already reports to %', NEW.reports_to,
NEW.emp_id;
    END IF;
    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_validate_manager ON employee;
CREATE TRIGGER trg_validate_manager
BEFORE INSERT OR UPDATE OF reports_to ON employee
FOR EACH ROW
EXECUTE FUNCTION trgfn_validate_manager();
```

Assigning a resigned employee as manager, and creating a reporting cycle (Aubrey Graham to report to his own report LeBron James):

ERROR: Manager 19 is not an active employee

ERROR: Circular reporting: employee 6 already reports to 4

Trigger 4 – trg_employee_exit

```
-- ------------------------------------------------------------------
-- TRIGGER 4: trg_employee_exit   (AFTER UPDATE OF status ON employee)
--   When an employee Resigns or is Terminated:
--     * their direct reports are moved to the leaving employee's manager
--     * they are removed as head of any department
--     * their pending leave requests are rejected
-- ------------------------------------------------------------------
CREATE OR REPLACE FUNCTION trgfn_employee_exit()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.status IN ('Resigned', 'Terminated')
       AND OLD.status NOT IN ('Resigned', 'Terminated') THEN

        UPDATE employee
           SET reports_to = NEW.reports_to
         WHERE reports_to = NEW.emp_id;

        UPDATE department
           SET head_emp_id = NULL
         WHERE head_emp_id = NEW.emp_id;
```

```
        UPDATE leave_request
           SET status = 'Rejected'
         WHERE emp_id = NEW.emp_id AND status = 'Pending';
    END IF;
    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_employee_exit ON employee;
CREATE TRIGGER trg_employee_exit
AFTER UPDATE OF status ON employee
FOR EACH ROW
EXECUTE FUNCTION trgfn_employee_exit();
```

Michael Jordan (employee 5, Sales Manager and head of Sales) resigns:

```
UPDATE employee SET status = 'Resigned' WHERE emp_id = 5;
```

Before – his direct reports and the Sales department head:

emp_id	employee	reports_to
12	PV Sindhu	5
13	Virat Kohli	5
14	Naomi Osaka	5

dept_name	head_emp_id
Sales	5

After – the trigger moved his team to his own manager (Sachin Tendulkar, employee 1) and cleared the department head:

emp_id	employee	reports_to
12	PV Sindhu	1
13	Virat Kohli	1
14	Naomi Osaka	1

dept_name	head_emp_id
Sales	NULL

8.5 Calling Procedures from the Application

The web application talks to the database through Supabase's REST interface (PostgREST), which can call SQL **functions** over HTTP but not procedures. Thin SECURITY DEFINER wrapper functions therefore check the caller's role and then CALL the procedure. Row Level Security policies decide which rows each role may read or change; they are defined in `sql/06_supabase_auth_rls.sql`.

```
CREATE OR REPLACE FUNCTION public.api_generate_payroll(p_month INT, p_year INT)
RETURNS INT
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public
AS $$
DECLARE v_count INT := 0;
BEGIN
    IF NOT app_is_staff() THEN
        RAISE EXCEPTION 'Only Admin or Payroll staff can generate payroll';
    END IF;
    CALL generate_monthly_payroll(p_month, p_year, v_count);
    RETURN v_count;
END;
$$;

-- Row Level Security example: employees see only their own payslips
CREATE POLICY pay_read ON payroll FOR SELECT TO authenticated
    USING (app_is_staff() OR emp_id = app_emp_id());
```

9. Real-World Transaction Demonstrations

9.1 Application and Database Connectivity

The application is a React single-page app. It connects to the PostgreSQL database with the supabase-js client: SELECT/INSERT/UPDATE/DELETE requests are sent to PostgREST, procedures are reached through the api_* functions (RPC), and every request carries the signed-in user's token so Row Level Security applies. Every screen has a "DB activity" panel that lists, for each user action, the SQL or PL/pgSQL component executed, the tables involved and the result – the flow the guidelines ask for:

User operation → SQL / PL/pgSQL → Database processing → Database update / retrieval → Result

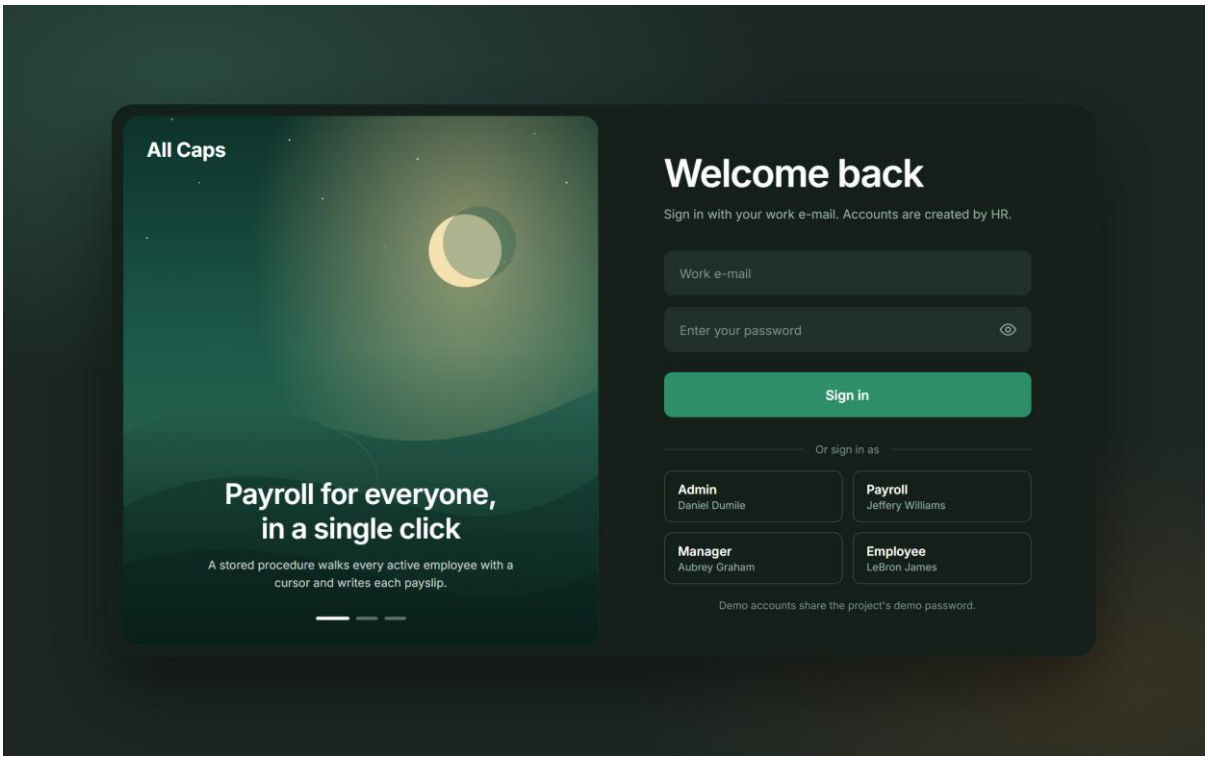


Figure 3 – Sign-in page. Each demo account opens a different role.

Row Level Security was verified by signing in as each role and counting the rows the database returned for the same queries:

Signed in as	employee	payroll	leave_request	attendance	Generate payroll?
Not signed in	denied	denied	denied	denied	denied
Admin (Daniel Dumile)	20	39	13	971	allowed
Payroll (Jeffery Williams)	20	39	13	971	allowed
Manager (Aubrey Graham)	8 (self, manager, team)	2 (own)	6 (own + team)	321	blocked
Employee (LeBron James)	2 (self + manager)	2 (own)	2 (own)	50 (own)	blocked

T1. Employee checks in

Question	Answer
Operation	LeBron James clicks “Check in” on his dashboard.
Tables involved	ATTENDANCE
SQL component	INSERT INTO attendance (emp_id, att_date, check_in, status) VALUES (6, CURRENT_DATE, <time>, 'Present'); – allowed by an RLS policy only for the employee’s own row and only for today.
Effect on database	One attendance row is inserted. UNIQUE (emp_id, att_date) rejects a second check-in on the same day.
Result	The dashboard shows “Present · In 16:02” and a Check out button (which performs an UPDATE of check_out).

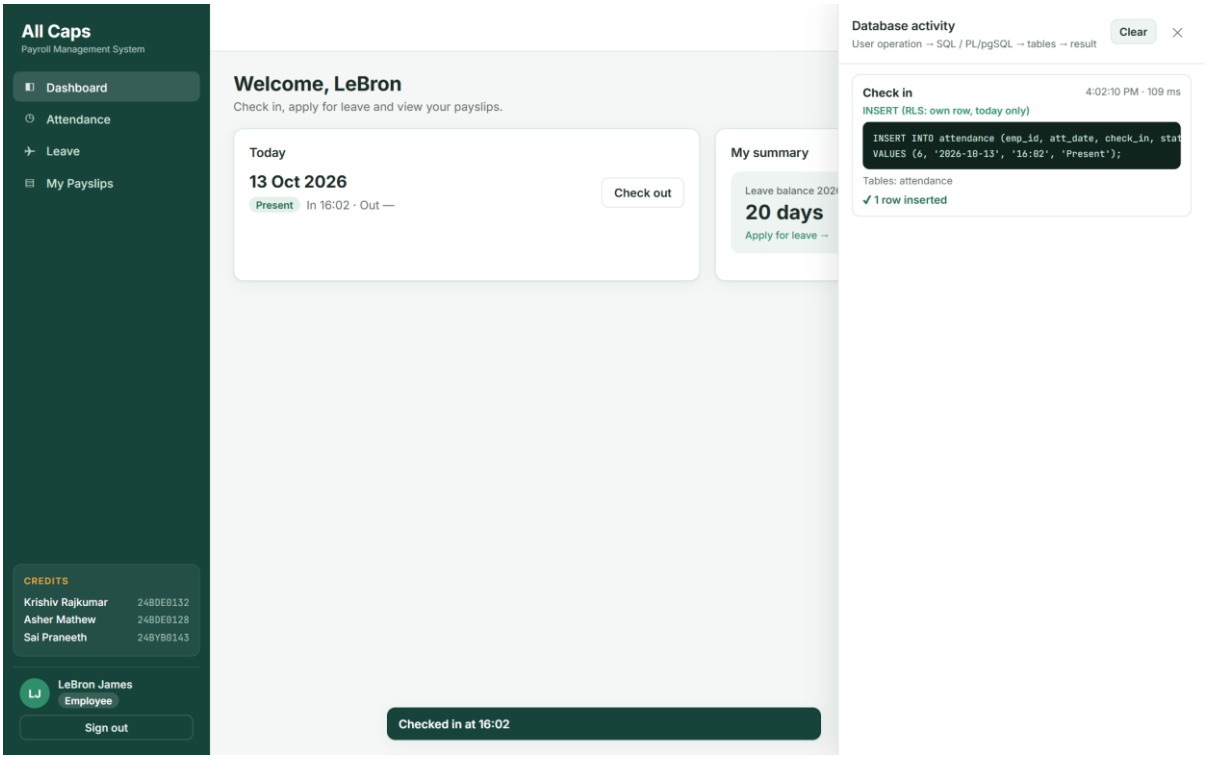


Figure 4 – Employee check-in with the INSERT shown in the DB activity panel.

T2. Employee applies for leave

Question	Answer
Operation	LeBron applies for 2 days of Casual leave (27–28 October).
Tables involved	LEAVE_REQUEST (read: LEAVE_REQUEST, DESIGNATION for the balance)
SQL component	SELECT fn_leave_balance(6, 2026) to show the balance; INSERT INTO leave_request … with status DEFAULT 'Pending'.
Effect on database	A Pending request is inserted. CHECK (end_date ≥ start_date) and the RLS policy (own row, status Pending, no approver) are enforced.
Result	The request appears under “My requests” with status Pending.

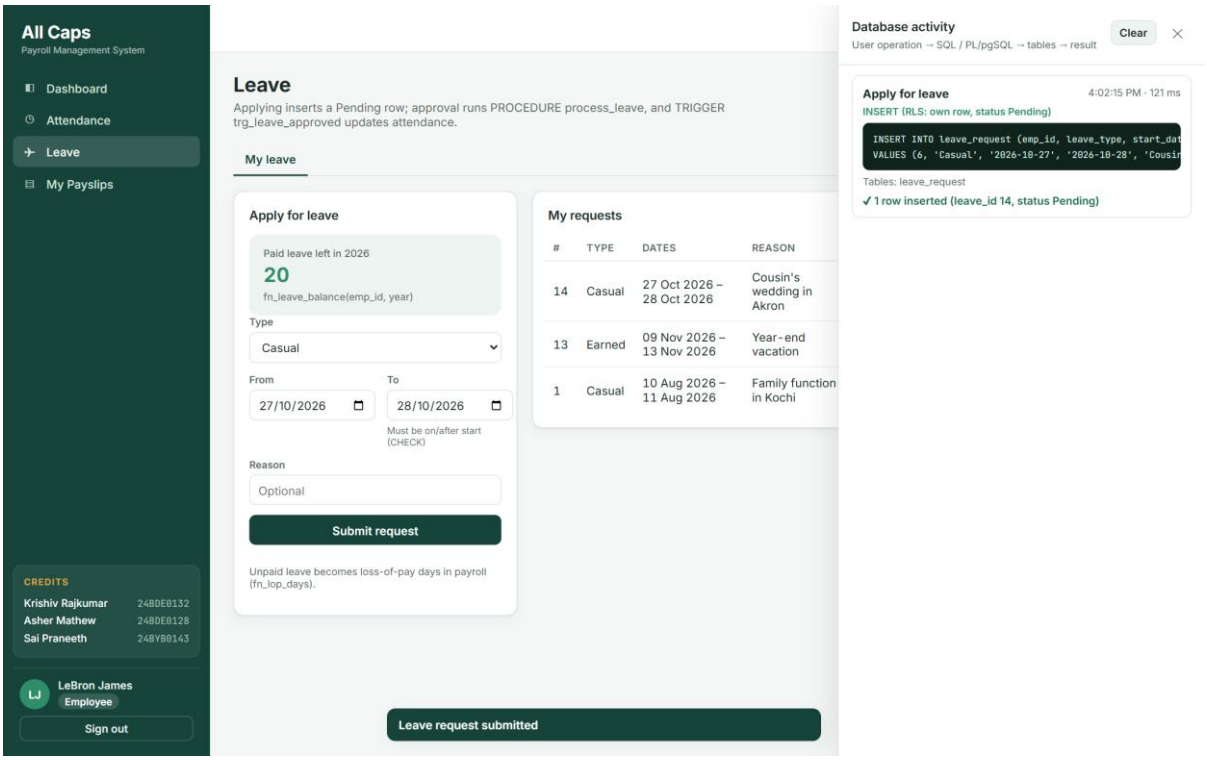


Figure 5 – Applying for leave: fn_leave_balance shown on the left, the INSERT in the activity panel.

T3. Manager approves leave

Question	Answer
Operation	Aubrey Graham approves Coco Gauff's sick leave (request #10).
Tables involved	LEAVE_REQUEST, EMPLOYEE, DESIGNATION, ATTENDANCE
SQL / PL/pgSQL component	SELECT api_process_leave(10, 'Approved') → CALL process_leave(10, 4, 'Approved') → fn_leave_balance → TRIGGER trg_leave_approved.
Effect on database	leave_request #10 becomes Approved with approved_by = 4; the trigger inserts 'Leave' attendance rows for 15 and 16 October.
Result	The request leaves the approval queue; Coco's attendance shows the two leave days (section 8.4).

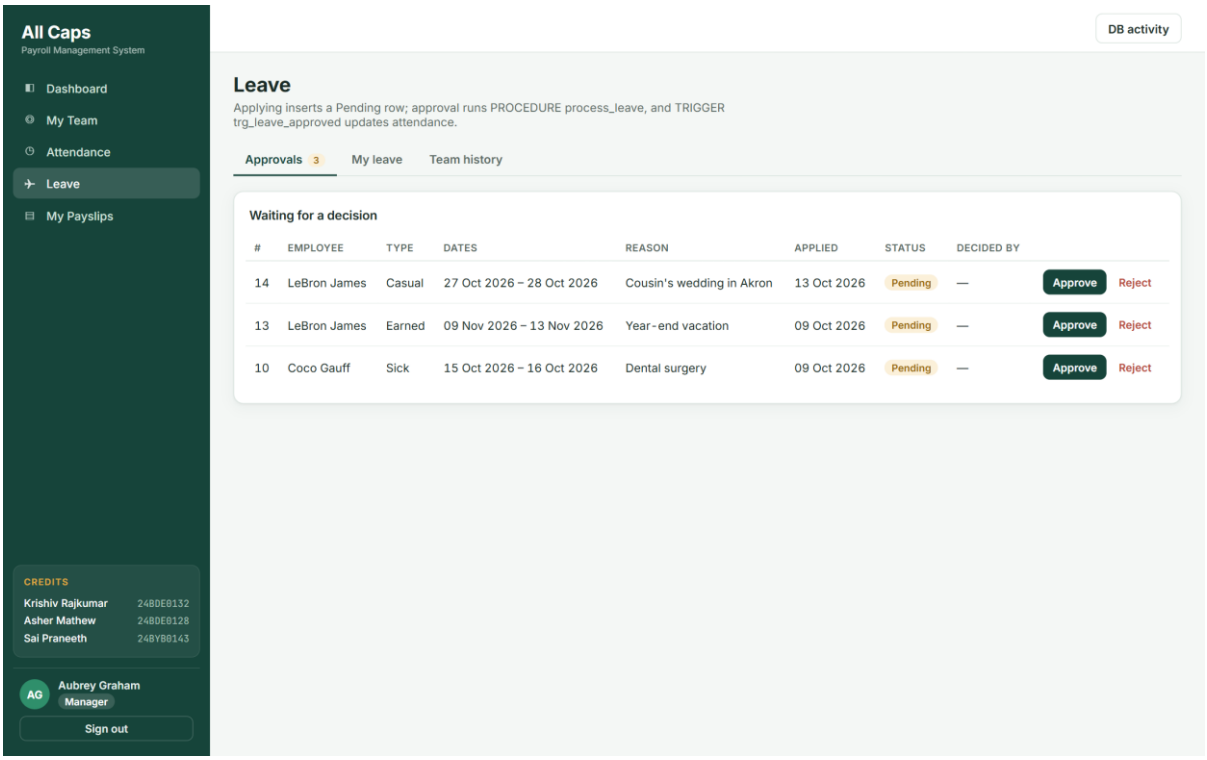


Figure 6 – The manager's approval queue (RLS returns only the manager's team).

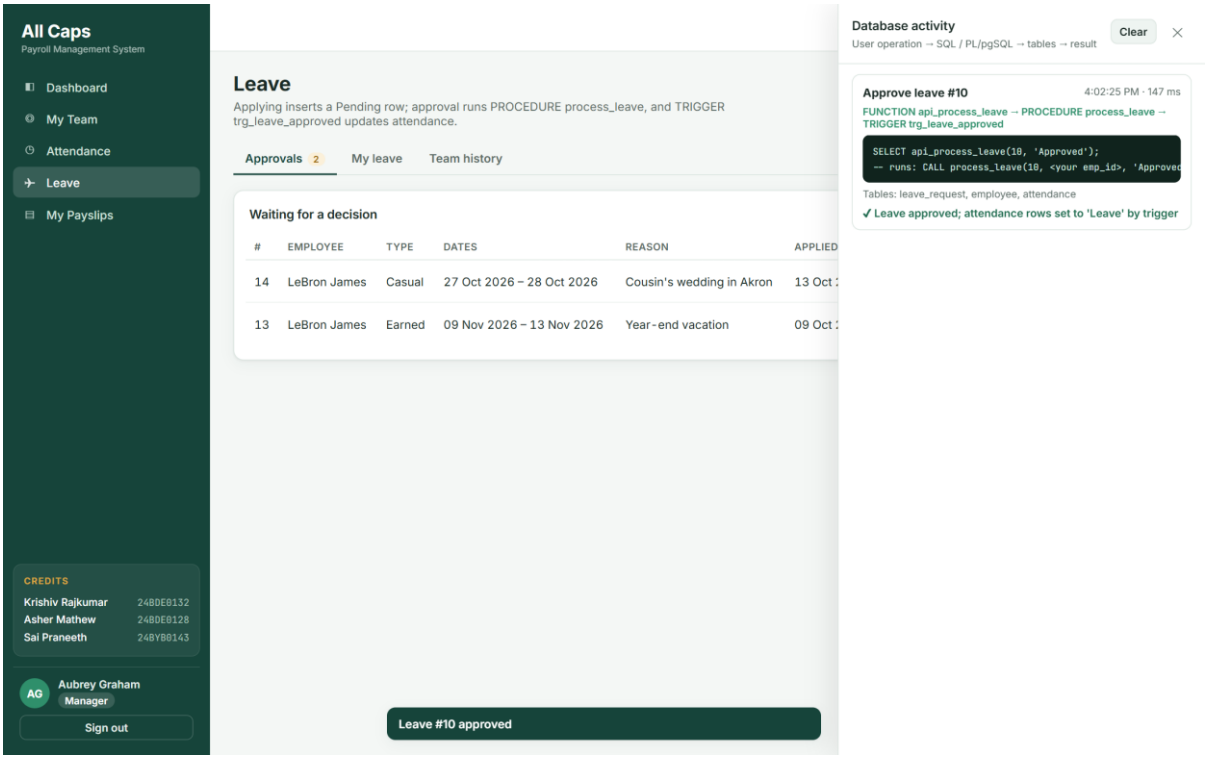


Figure 7 – After approval: the procedure and trigger chain shown in the activity panel.

T4. Payroll officer generates the monthly payroll

Question	Answer
Operation	Jeffery Williams selects October 2026 and clicks "Generate payroll".
Tables involved	EMPLOYEE, DESIGNATION, ATTENDANCE, LEAVE_REQUEST, SALARY_COMPONENT (read); PAYROLL, PAYROLL_DETAIL (write)

Question	Answer
SQL / PL/pgSQL component	SELECT api_generate_payroll(10, 2026) → CALL generate_monthly_payroll(10, 2026) – cursor over active employees using fn_working_days, fn_lop_days and fn_component_amount.
Effect on database	19 PAYROLL rows (status Generated) and one PAYROLL_DETAIL row per active salary component for each are inserted. Running it again inserts nothing.
Result	The payroll table lists all 19 active employees with LOP days, gross, deductions and net pay; totals are shown above it.

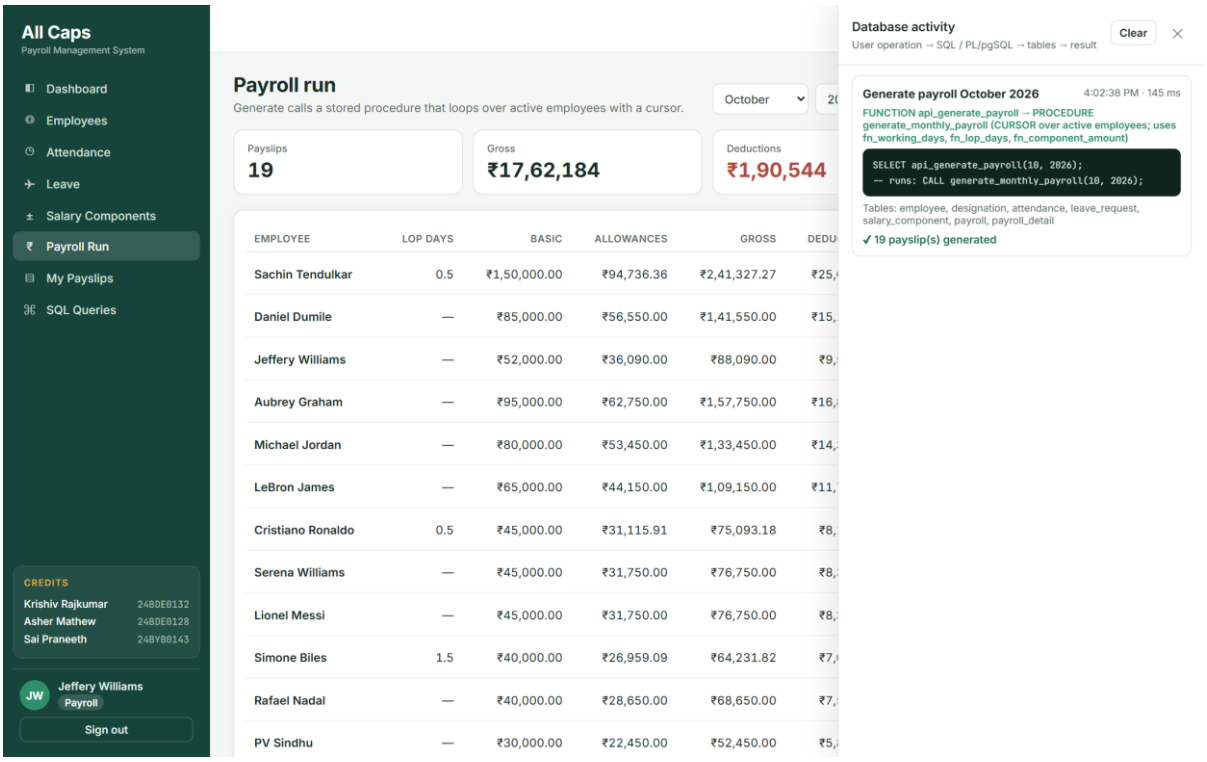


Figure 8 – Generating October payroll: one click runs the cursor-based procedure for 19 employees.

T5. Viewing an itemised payslip

Question	Answer
Operation	The payroll officer (or the employee, from My Payslips) opens a payslip.
Tables involved	PAYROLL, PAYROLL_DETAIL, SALARY_COMPONENT, EMPLOYEE, DEPARTMENT, DESIGNATION
SQL component	SELECT … FROM payroll_detail JOIN salary_component … WHERE payroll_id = ? (the M:N bridge table) plus the employee details.
Effect on database	Read only. RLS lets an employee read only their own payslips.
Result	Earnings (basic, loss of pay, allowances), deductions and net pay, ready to print.

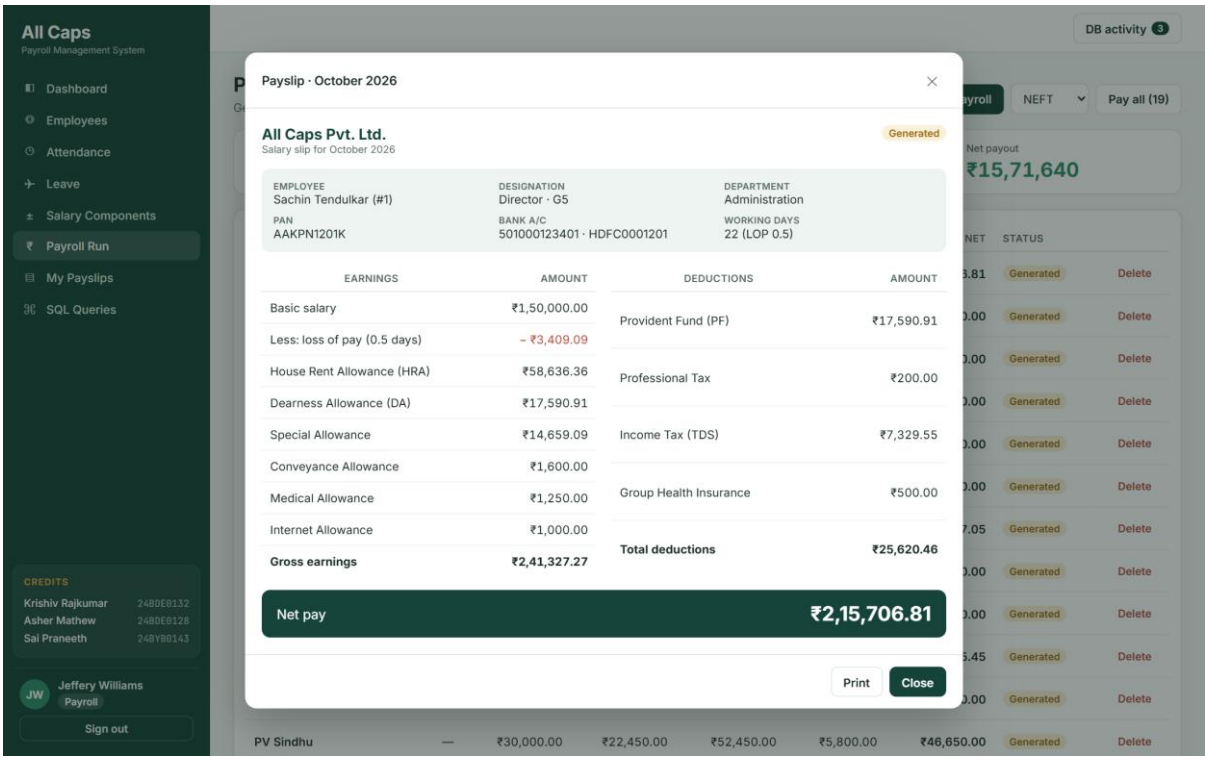


Figure 9 – Itemised payslip built from PAYROLL_DETAIL.

T6. Paying the month's salaries

Question	Answer
Operation	The payroll officer chooses NEFT and clicks "Pay all".
Tables involved	PAYROLL
SQL / PL/pgSQL component	SELECT api_pay_payroll(10, 2026, 'NEFT') → CALL pay_monthly_payroll(…) – cursor FOR UPDATE with UPDATE … WHERE CURRENT OF.
Effect on database	Every Generated row of the month becomes Paid with payment_date, payment_mode and a unique transaction_ref (CHECK: Paid ⇒ date and mode present).
Result	"19 payroll row(s) marked Paid"; the rows are now locked by trg_lock_paid_payroll.

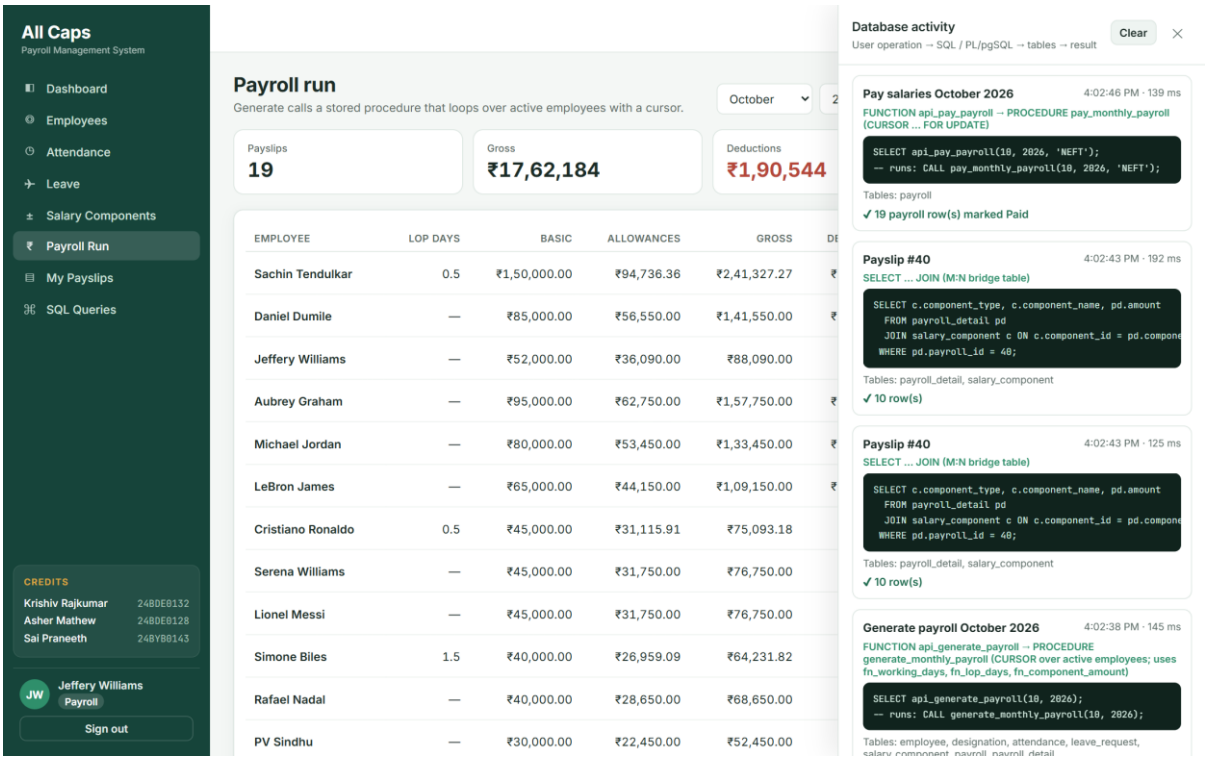


Figure 10 – Paying salaries through the second cursor-based procedure.

T7. Attempt to delete a paid payslip

Question	Answer
Operation	The payroll officer clicks Delete on Sachin Tendulkar's paid October payroll.
Tables involved	PAYROLL (PAYROLL_DETAIL would cascade)
SQL / PL/pgSQL component	DELETE FROM payroll WHERE payroll_id = 40; → BEFORE DELETE trigger trg_lock_paid_payroll.
Effect on database	The trigger raises an exception, so the DELETE is rolled back and nothing changes.
Result	The error message from the database is shown to the user.

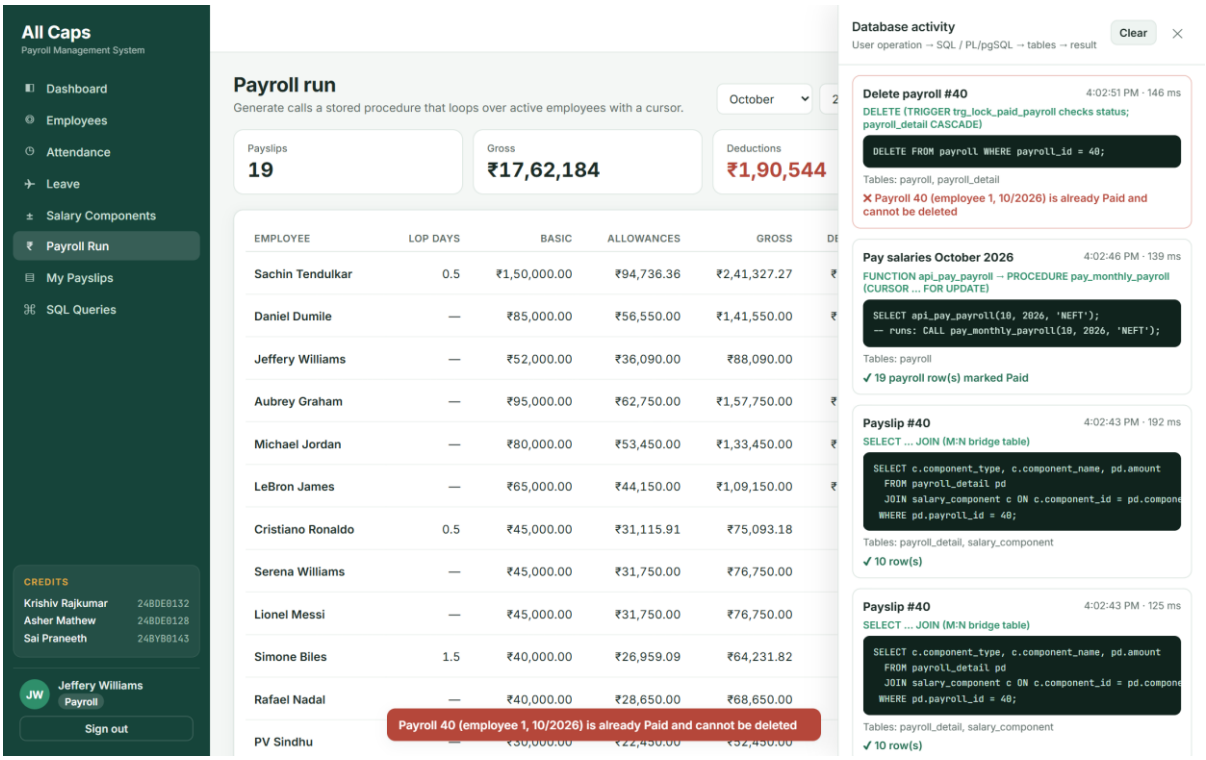


Figure 11 – The trigger blocks deleting a paid payroll; the error comes from the database.

T8. Employee resignation

Question	Answer
Operation	The Admin edits Michael Jordan's record and sets status to Resigned.
Tables involved	EMPLOYEE, DEPARTMENT, LEAVE_REQUEST
SQL / PL/pgSQL component	UPDATE employee SET status = 'Resigned' WHERE emp_id = 5; → AFTER UPDATE trigger trg_employee_exit (which in turn fires trg_validate_manager for the moved employees).
Effect on database	His three reports now report to Sachin Tendulkar, the Sales department has no head, and any pending leave of his is rejected. He is excluded from later payroll runs.
Result	Shown with before/after outputs in section 8.4 (Trigger 4).

T9. Running the report queries from the application

Question	Answer
Operation	The Admin opens SQL Queries and runs Q4 (department-wise salary cost).
Tables involved	PAYROLL, EMPLOYEE, DEPARTMENT (through view q04_department_salary_cost)
SQL component	SELECT * FROM q04_department_salary_cost; – a view created WITH (security_invoker = true) so RLS still applies.
Effect on database	Read only.
Result	The SQL text, the concepts used and the live result table are shown on one page.

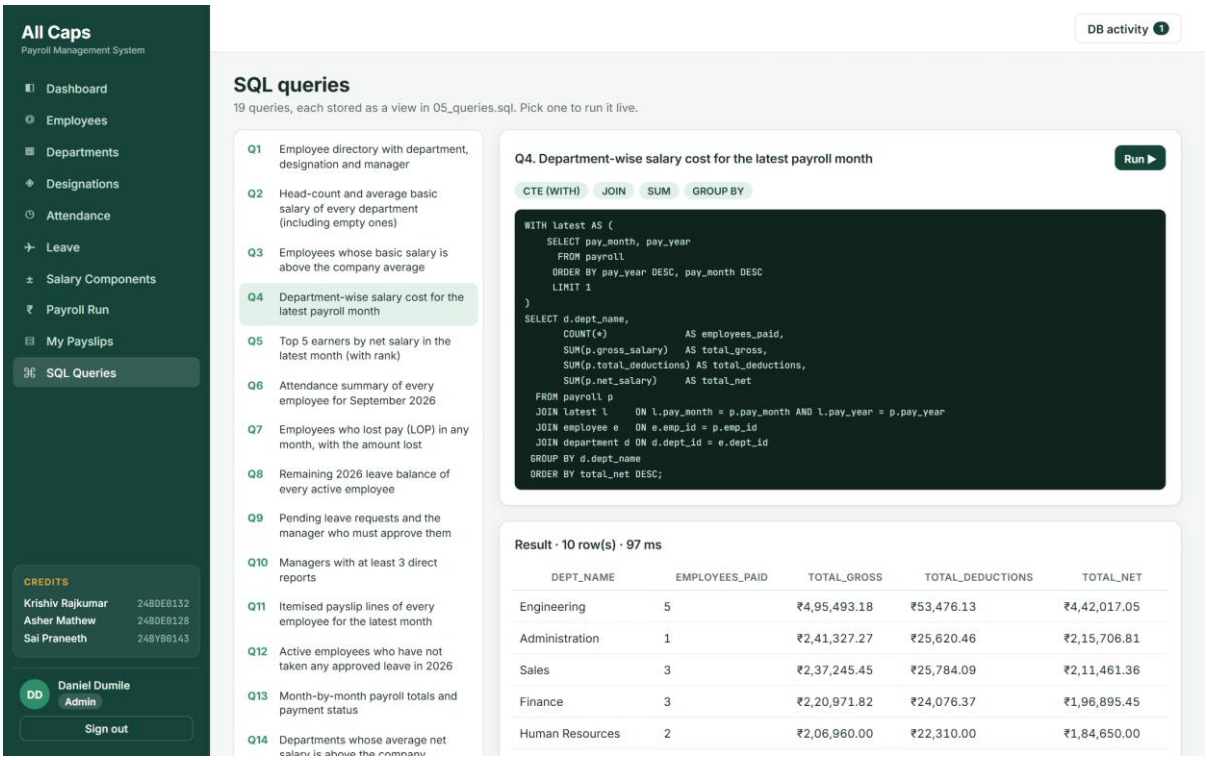


Figure 12 – The SQL Queries page running Q4 live against the database.

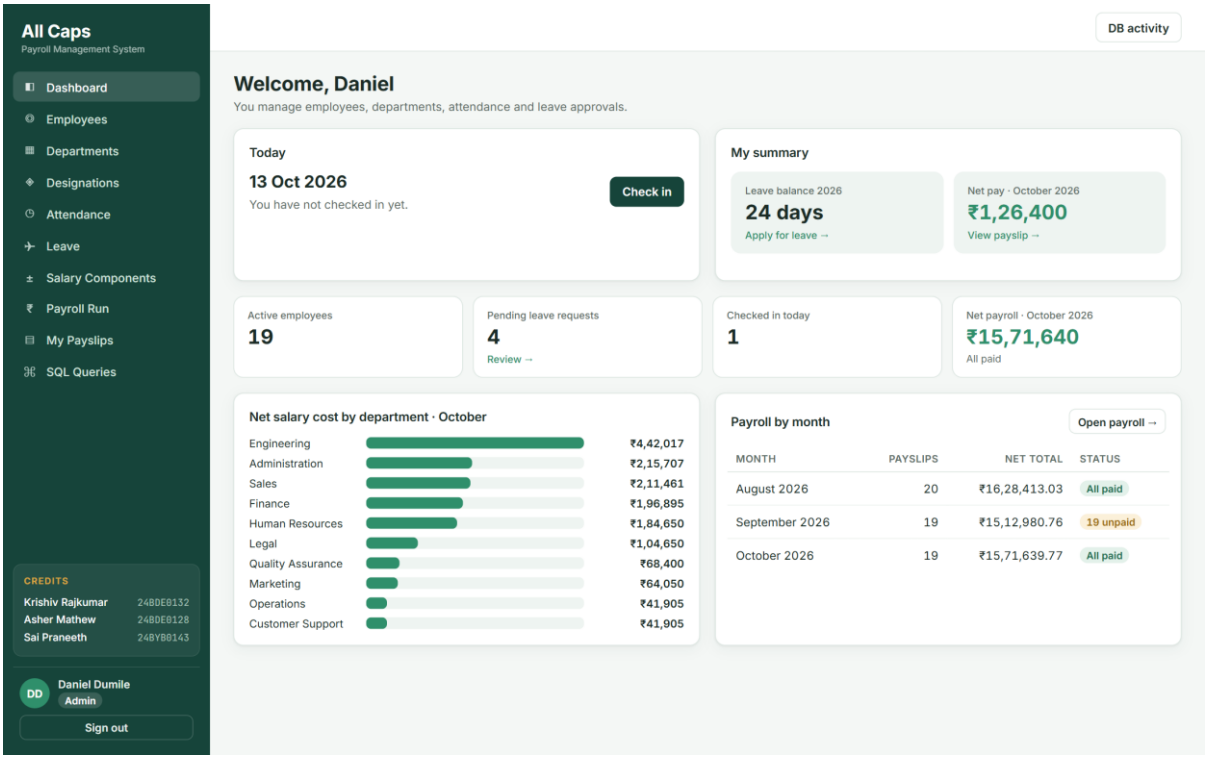


Figure 13 – Admin dashboard built from views q13 and q04 and live counts.

10. Conclusion

The Employee Payroll Management System has been implemented as a PostgreSQL database on Supabase with a role-based web application. The database has 8 normalised tables with complete primary key, foreign key, NOT NULL, UNIQUE, CHECK and DEFAULT constraints, populated with realistic data for a 20-employee company over three months.

Nineteen SQL queries cover joins, self-joins, aggregation, grouping with HAVING, correlated and scalar subqueries, set operations, window functions and date functions. Four functions, three procedures (two using explicit cursors) and four triggers implement the real business operations – loss-of-pay calculation, leave approval, payroll generation, salary payment and employee exit – and the database itself rejects invalid operations such as editing a paid payslip, approving one's own leave or exceeding the leave balance.

Every component has a clear purpose in the application: each user action in the web app maps to a specific SQL statement, procedure or trigger, and the DB activity panel shows that flow live during the demonstration. Possible extensions are tax-regime based TDS calculation, salary revisions with effective dates, and an audit log table populated by triggers.